

Table of Contents

Sage200APIMicroservice — Technical Project Review & Working Protocol	5
1. Project Overview	5
High-Level Integration Flow.....	5
Data-Flow Summary (End-to-End Integration Path).....	6
Error & Replay Behaviour	7
Core Design Objectives	9
2. System Architecture	9
Core Components	9
Simplified Internal Database Diagram	11
Architecture Principles	11
3. Folder & File Structure	11
Solution Overview	12
Detailed Folder Breakdown.....	12
Cross-Project Dependencies	14
4. API Access & Validation	14
Required Request Metadata.....	15
Validation Logic (Unified Across HTTP & Kafka)	15
Kafka Message Header Equivalents.....	16
Rejection Workflow	16
Database Entities Involved.....	17
Security & Observability Notes	17
5. Kafka Topics & Message Workflow	17
5.1 Topic Catalogue.....	17
5.2 Required Kafka Headers	18
5.3 Inbound Payload Schemas	19
5.4 Customer Account Code Generation Logic	21
5.5 Outbound Result Envelope	22
5.6 Processing Flow (Inbound → Outbound)	22
5.7 Payment Export (Airflow Integration)	23
5.8 Compliance with Sage OpenAPI	23
6. HTTP Endpoints & Airflow Workflow	24
6.1 Overview of API Layer	24
6.2 Shared Headers for HTTP Endpoints	24
6.3 Payment & Allocation Export Flow	25
6.4 Diagnostic Endpoints	26
6.5 Allocation Workflow (Detail)	27
6.6 Error Handling & Logging	27

6.7 Sage OAuth Integration (Quick Overview)	28
6.8 Endpoint Workflow Diagram	28
6.9 Audit & Metrics Summary	28
Summary of Section 6	29
7. Kafka Contracts & Orchestrations	29
7.1 Topic Naming Convention (underscore style)	30
7.2 Common Envelope Structure (for Inbound Messages)	30
7.3 Entity-Specific Schemas	31
7.4 Outbound Result Contracts	32
7.5 Idempotency & Correlation Rules	33
7.6 Consumer Behaviour & DLQ Policy	33
7.8 Orchestration Flow Diagram	34
Summary of Section 7	34
7A. Kafka Schema References (Developer Appendix)	34
7A.1 Common Envelope Schema	35
7A.2 Customer Payload (MDM_CUSTOMER)	36
7A.3 SOP Order Payload (MDM_SOP_ORDER)	36
7A.4 Invoice Payload (MDM_INVOICE)	37
7A.5 Payment Result Envelope (Outbound)	38
Developer Notes	39
8. Internal Database Design & Mapping Logic	40
8.1 Overview	40
8.2 Core Tables	40
8.3 Entity Relationships (Simplified Diagram)	41
8.4 TransactionAttempts Flow	42
8.5 ExternalIdLink Logic	42
8.6 AuditLog Usage	43
8.7 Data Retention and Purging Strategy	43
8.8 Database Integrity Rules	44
8.9 Example Data Lineage (End-to-End)	44
Summary of Section 8	44
9. Service Layer & Orchestrator Responsibilities	45
9.1 Layer Overview	45
9.2 Orchestrator Role	45
9.3 Domain Services	46
9.4 Service Interaction Diagram (Invoice Flow)	48
9.5 Database Integration (per orchestration)	48
9.6 Resilience & Retry	48
9.7 Sage API Integration Rules	49

9.8 Orchestrator Error Strategy.....	49
9.9 Example Orchestrator Pseudocode	49
9.10 Benefits of the Orchestration Model.....	50
Summary of Section 9.....	50
10. Error Handling, Dead-Letter Queue (DLQ), and Retry Policies.....	50
10.1 Error-Handling Philosophy.....	50
10.2 Error Classification	51
10.3 Retry Policy.....	51
10.4 Dead-Letter Queue (DLQ).....	51
10.5 Audit and Visibility Integration	52
10.6 Typical Error Scenarios.....	53
10.7 Logging & Monitoring	53
10.8 Error Workflow Diagram (Conceptual).....	54
10.9 Developer Guidelines	54
Summary of Section 10.....	54
11. Security, API Key Management & OAuth Governance	54
11.1 Security Architecture Overview	55
11.2 API Key Lifecycle	55
11.3 Header-Based Authorization Context.....	56
11.4 Sage OAuth 2.0 Token Governance	57
11.5 Security & Audit Integration	58
11.6 Data Protection & Secrets Management.....	58
11.7 Multi-Tenant Safety	58
11.8 Security Diagram (Conceptual).....	59
11.9 Security Failure Handling.....	59
Summary of Section 11	59
12. Monitoring, Observability & Metrics.....	60
12.1 Observability Objectives.....	60
12.2 Metrics Pipeline Overview	60
12.3 Prometheus / Grafana Integration.....	61
12.4 Business Dashboard (HTML + JS Integration).....	61
12.5 Log Correlation & Tracing	63
12.6 Automated Alerts	63
12.7 Health & Status API Responses.....	63
12.8 Metric Visualization Flow.....	64
12.9 Security & Access Control	64
Summary of Section 12.....	64
13. Data Governance & Compliance	64
13.1 Governance Framework Overview	65

13.2 Data Ownership & Tenancy.....	65
13.3 Data Retention Policies	66
13.4 GDPR & Data Privacy Alignment.....	66
13.5 Encryption & Data Security	66
13.6 Access Control & Roles	67
13.7 Data Lineage & Traceability	67
13.8 Tenant Offboarding & Data Deletion.....	68
13.9 Compliance Reporting	68
13.10 Data Governance Diagram (Conceptual).....	69
Summary of Section 13.....	69
14. MVP Completion Plan & Next Steps.....	69
14.1 MVP Objectives Recap	70
14.2 Remaining MVP Tasks	70
14.3 Validation & UAT Plan.....	71
14.4 Deployment Plan	72
14.5 Monitoring & Handover.....	73
14.6 Future Roadmap (Post-MVP)	73
14.7 Summary Timeline	73
14.8 MVP Success Criteria	74
Summary of Section 14.....	74

Sage200APIMicroservice — Technical Project Review & Working Protocol

Version: 1.0 (MVP Phase)

Date: October 2025

Author: Stephen Brett

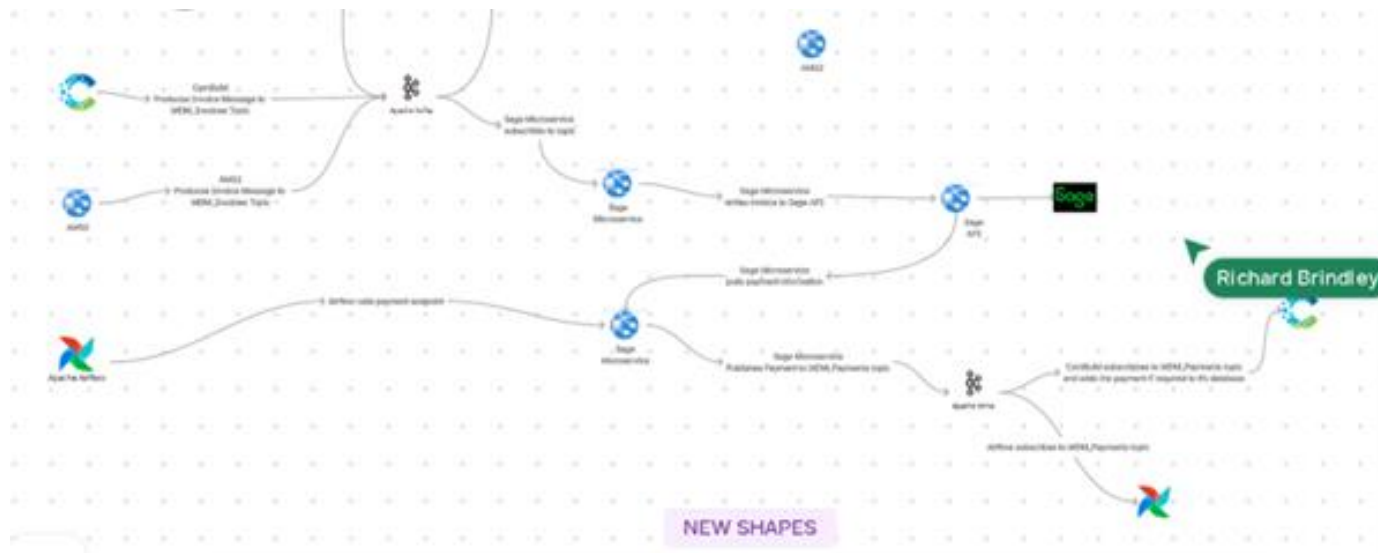
1. Project Overview

The **Sage200 API Microservice** (Sage200APIMicroservice) is a secure integration gateway connecting multiple external business systems—initially developed with **CymBuild** as the reference implementation—to the **Sage 200 API**.

Although CymBuild is the first adopter, the architecture is intentionally **multi-tenant**, allowing any authorised external system to integrate via the same Kafka-based MDM (Message Data Management) layer.

The microservice manages and synchronises **Customers**, **Sales Orders (SOP)**, **Invoices**, and **Payments** between external systems and Sage 200, while ensuring data integrity, replay safety, and full auditability.

High-Level Integration Flow



Data-Flow Summary (End-to-End Integration Path)

Lane	Step	Description	Message / Action	Notes
1 – Calling App	1. Publish Invoice Message	Sends a new invoice transaction to Kafka topic MDM_INVOICE.	JSON payload + headers → (SiteName, CompanyID, Key, ExternalRef).	Key identifies the authorised ApiKeys record.
	2. Listen for Results	Subscribes to MDM_INVOICE_RESULTS.	Receives success / failure event.	Used to update local state.
2 – Kafka MDM	3. Route Inbound Message	Delivers message to the subscribed microservice consumer group.	Kafka → Sage Microservice.	MDM cluster is externally managed.
3 – Sage Microservice	4. Consume & Validate	Reads message, validates ApiKey and context (SiteName/CompanyID).	Reject if invalid → DLQ; else continue.	Logs TransactionAttempt + AuditLog entry.
	5. Resolve Customer	Checks ExternalIdLinks for CustomerCode/SageCode.	Upsert Customer via Sage API if missing or outdated.	Stores new Sage URN/ID.
	6. Create SOP Order	Builds Sales Order from payload.	POST → Sage API SOP Order endpoint.	Maps order lines and totals.
	7. Generate Invoice	Creates invoice linked to order and customer.	POST → Sage API Invoice endpoint.	Excludes document rendering.
	8. Persist and Publish Result	Writes audit records → DB (TransactionAttempts, AuditLogs, ExternalIdLinks).	Emits MDM_INVOICE_RESULTS with summary.	Offset committed after success.
4 – Sage 200 API	9. Process Requests	Executes create/update operations for Customer, Order, Invoice.	Returns Sage IDs and URNs.	OAuth secured per company.
5 – Apache Airflow	10. Scheduled Payment Sync	Timer job queries Sage for new payments and allocations.	GET → Sage API / Payments.	Auth context per tenant.
	11. Publish Payments	Sends updates to Kafka MDM_PAYMENTS topic.	JSON payload with ExternalRef and Sage URN.	Used for reconciliation.
	12. Reconcile Payments	Consumes MDM_PAYMENTS topic for its ApiKeyId.	Updates local ledger allocations.	Completes payment loop.

Error & Replay Behaviour

- **Permanent failure:** message moved to *_DLQ topic with reason and correlation ID.
- **Transient failure:** consumer retries before commit.
- **Replay:** archived message can be replayed by reseeding the Kafka offset or re-publishing the original payload (idempotent check prevents duplicates).

1. Calling Application (e.g., CymBuild)

- Publishes messages such as MDM_INVOICE into the **MDM Kafka cluster**.
- Each message contains both transactional data and routing metadata used by the microservice to determine context and authorisation.
- The standard message envelope is defined as:
 - (SiteName, CompanyID, Key, ExternalRef)
 - **SiteName** – the Sage site context (optional; defaults from *appsettings.json*).
 - **CompanyID** – the Sage company context (optional; defaults from *appsettings.json*).
 - **Key** – the API Key string issued to the calling application; mapped to the corresponding ApiKeys.Id record in the internal database.
 - **ExternalRef** – the calling application’s unique identifier for the record or transaction (for example, its own Customer URN or Invoice GUID).
- These values allow the microservice to link external records with their Sage equivalents through the ExternalIdLinks table.

2. Kafka (MDM Cluster)

- Provides the asynchronous transport layer.
- Owns all inbound topics (for example MDM_INVOICE, MDM_CUSTOMER) and outbound result topics (for example MDM_INVOICE_RESULTS, MDM_PAYMENTS).
- The Sage Microservice subscribes as a consumer but does **not** host Kafka; it simply connects to the managed cluster within the MDM environment.

3. Sage 200 API Microservice

- **Payload Translation and Processing**
 - When an MDM_INVOICE message is consumed, the service first validates the header tuple (SiteName, CompanyID, Key, ExternalRef).
 - SiteName and CompanyID default from configuration if omitted.

- Key is looked up in the ApiKeys table; if the key exists, is active, and not expired, processing continues; otherwise the request is rejected but still logged in AuditLog.
- **Customer Validation and Upsert Logic**
 - The service inspects the invoice payload to identify the customer record.
 - If a **CustomerCode** (external system's unique reference) exists, it checks ExternalIdLinks for a corresponding Sage URN/Customer Code.
 - If found, the existing Sage record is used.
 - If not found, or if the payload includes updates, the customer is created or updated (UPSERT) in Sage via the Sage API, then the resulting Sage URN / Customer Code is stored in ExternalIdLinks for future correlation.
 - Alternatively, if the payload already provides a **SageCode**, the microservice treats it as a direct link to the existing Sage record.
- **Order and Invoice Creation**
 - Once the customer is confirmed, the SOP Order data within the payload is extracted and sent to the appropriate Sage endpoint to create the Sales Order.
 - The invoice component (identified by InvoiceNumber in the payload) is then generated from the newly created Sales Order through the Sage API assigning this Invoice Number.
 - Document generation (PDF rendering, email, etc.) is intentionally excluded from the microservice workflow and handled externally by the calling system.
- **Persistence and Audit**
 - All key events—validations, API calls, responses, and outcomes—are written to the internal database:
 - TransactionAttempts tracks execution attempts and statuses.
 - ExternalIdLinks maintains cross-reference mappings between external and Sage records.
 - AuditLog records full contextual detail for traceability.
 - This persistence guarantees replay capability and end-to-end accountability.
- **Result Publication**
 - After a successful Sage transaction, the microservice commits the Kafka offset and publishes a confirmation event (for example MDM_INVOICE_RESULTS) back into Kafka.
 - These results are consumed by the originating calling application for reconciliation and by the microservice itself for internal auditing.

4. Sage 200 API (OAuth Secured)

- All interactions use OAuth 2.0 for token-based authentication.
- The API performs the requested create/update actions for Customers, SOP Orders, and Invoices, returning official Sage identifiers which are written to the ExternalIdLinks table.

5. Payment and Allocation Flow

- All payment and allocation activity originates **inside Sage 200**.
- The microservice periodically queries Sage for new payments and allocations.
- For each entry, it resolves the related calling application through its stored ApiKeys.Id and ExternalIdLinks.ExternalRef.
- It then publishes payment information to Kafka (MDM_PAYMENTS), allowing CymBuild—or any other authorised system—to consume these updates and reconcile them against its own local transactions.

Core Design Objectives

- **Reliable and Idempotent Processing** – Each inbound message is processed exactly once through database-level idempotency tracking.
- **Secure Multi-Tenant Access** – ApiKeys and per-company context ensure isolated and auditable access for each calling application.
- **Full Traceability** – Every Kafka message and Sage API interaction is recorded for replay or forensic analysis.
- **Automated Payment Reconciliation** – Apache Airflow triggers scheduled payment-synchronisation jobs that pull allocations from Sage 200 and republish them through Kafka for external systems.

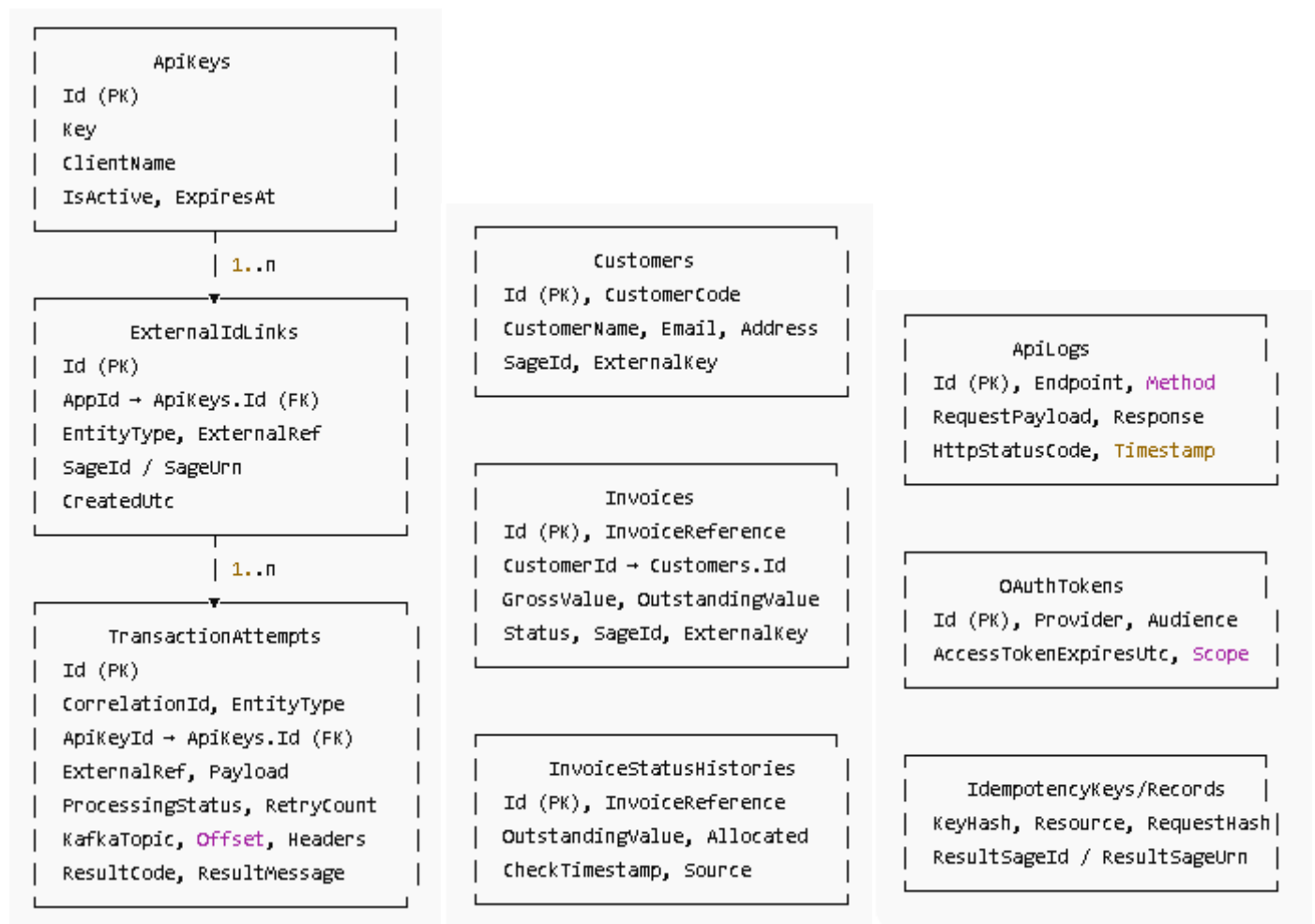
2. System Architecture

Core Components

Layer	Description
CymBuild (or other Calling Applications)	External business system(s) that generate operational data — customers, SOP orders, invoices, and payments. CymBuild publishes events into the MDM Kafka cluster (e.g., MDM_INVOICE, MDM_CUSTOMER). Each message contains metadata (SiteName, CompanyID, Key, ExternalRef) used by the microservice for context, authorisation, and record correlation.

Layer	Description
Kafka Cluster (MDM)	Acts as the asynchronous message bus. Topics use underscore naming (e.g., MDM_INVOICE, MDM_CUSTOMER, MDM_INVOICE_RESULTS, MDM_PAYMENTS). Kafka ensures reliable delivery, replayability, and decoupling between producers (calling systems) and consumers (microservice). The Sage Microservice subscribes to the inbound topics and publishes to corresponding result topics.
Sage200 API Microservice	The core integration service that: • Consumes inbound Kafka messages. • Validates headers, API keys, and context (SiteName/CompanyID). • Translates payloads into Sage 200 OpenAPI requests using OAuth 2.0 tokens. • Manages idempotency, mapping, and auditing through its internal SQL Server database. • Publishes results or failures (*_RESULTS and *_DLQ topics) back into Kafka.
Sage 200 API (OAuth Secured)	The official Sage 200 REST API endpoints used for all Customer, SOP Order, Invoice, and Payment operations. Access is handled by per-tenant OAuth 2.0 tokens stored in the microservice's OAuthTokens table.
Internal Database (Sage200APIMSAAdvanced)	SQL Server database providing state persistence and full audit traceability. It tracks: • API key authorisations. • External↔Sage ID mappings. • Transaction attempts, retries, and durations. • Audit and API logs. • Invoice status history. • OAuth tokens and idempotency records. Used for replay safety and analytics.
Apache Airflow DAGs	External scheduler that invokes the microservice's /api/payments/export-jobs endpoint on a timer. Each DAG pulls new payments and allocations from Sage 200 via the API, updates local tables, and republishes payment messages (MDM_PAYMENTS) to Kafka for calling applications.

Simplified Internal Database Diagram



Architecture Principles

- **Isolation:** Each tenant (calling app) operates via its own ApiKey, isolating audit trails and data access.
- **Resilience:** Kafka back-pressure, DLQ topics, and TransactionAttempts retries ensure no data loss.
- **Traceability:** Every API call, Kafka event, and Sage transaction is captured and correlated via CorrelationId.
- **Security:** All API operations require a valid ApiKeys.Key and are logged with source IP and timestamp.
- **Extensibility:** Designed to support additional topics (e.g. MDM_PAYMENT_ALLOCATIONS, MDM_PRODUCT_UPDATES) without architecture changes.

3. Folder & File Structure



SageStructure.txt

Solution Overview

The Sage 200 API Microservice solution consists of three core projects that work together to deliver full integration between the calling applications (e.g. CymBuild), Kafka, and the Sage 200 API.

Project	Purpose / Responsibility
Sage200Microservice.API	ASP.NET Core Web API host. Exposes controllers, authentication, middleware, metrics, and Swagger docs. Handles inbound HTTP and health-check requests.
Sage200Microservice.Services	Business-logic and orchestration layer. Houses the Kafka consumers & producers, service implementations, DTO models, orchestration logic, and Sage API client integrations.
Sage200Microservice.Data	Entity Framework Core data-access layer. Defines database entities, repositories, and migrations for persistence (AuditLogs, ApiKeys, TransactionAttempts etc.).

Detailed Folder Breakdown

◆ Sage200Microservice.API

Folder	Purpose / Key Components
/Controllers/	REST controllers mapping to business entities — CustomersController, SalesInvoicesController, SopOrdersController, etc. These expose HTTP endpoints for admin and direct API operations.
/Attributes/	Custom attributes for request handling — e.g., SageRoutingHeadersAttribute ensures (SiteName, CompanyID, Key, ExternalRef) headers are validated.
/Configuration/	Application configuration: CORS, API key rotation, security headers, rate-limiting, IP filters.
/Middleware/	Cross-cutting behaviors — API key auth (ApiKeyAuthenticationMiddleware), auditing, tracing, and global exception handling.
/Metrics/	Prometheus metrics collectors (ApiMetrics, DatabaseMetrics, SageApiMetrics).
/HealthChecks/	Liveness/readiness probes — KafkaConsumerLivenessHealthCheck, SageApiHealthCheck.
/DTOs/	DTOs for request & response contracts (Invoice, Customer, Audit Log etc.).
/Extensions/	Service registration helpers (e.g., ServiceCollectionKafkaExtensions).

Folder	Purpose / Key Components
/Validators/	Fluent Validation rulesets for payloads and filters.
/OpenAPI/	Full Sage 200 OpenAPI definitions (sales.json, sop.json, stock.json, etc.).
/Documentation/	Internal technical references — process maps, flow docs, Sage API field mapping, PowerShell scripts.
/Logging/	Serilog and diagnostic startup configuration.
/Tracing/	OpenTelemetry sources for activity correlation.
/Swagger/	Swagger customisation (filters to ensure headers, security definitions).
/Monitoring/	Dashboard and alert services for runtime observability.
/wwwroot/	Static assets (e.g., embedded business metrics dashboard HTML + JS).

◆ Sage200Microservice.Services

Folder	Purpose / Key Components
/Implementations/	Concrete service logic — CustomerService, SalesInvoicesService, SopOrderService, ReconciliationService, OAuthTokenStore, etc.
/Interfaces/	Service contracts — ICustomerService, ISalesInvoicesService, ISopOrderService, IInvoiceRequestOrchestrator and others define abstractions for DI.
/Messaging/	Kafka integration layer: producers, consumers, options, and envelopes. • Consumers/ – inbound message handlers (SalesInvoiceCreateConsumer, ResultConsumers, DlqEnvelope). • Contracts/ – Kafka payload schemas (e.g., KafkaInvoiceCreateMessage). • Orchestration/ – business process logic (InvoiceRequestOrchestrator). • Requests/ – incoming request models from Kafka (MdmlInvoiceMessage).
/Models/	Rich domain and Sage DTO models used throughout services (Customer, SOP, Sales, Reconciliation, Sage entities).
/Http/	Outbound HTTP handlers and delegating handlers for OAuth, correlation IDs, and logging.
/Logging/	Persistent DB logging (DbLogWriter, AuditLogRecord), with optional AES-GCM field encryption (/Encryption/).
/Configuration/	Feature options and tuning (SopFeaturesOptions, background service settings).

Folder	Purpose / Key Components
/Shared/	Helper utilities (FriendlyFilters, Helpers, StatusMapping).
/Tracing/	TracingHelper for correlation across Kafka and Sage API requests.

◆ Sage200Microservice.Data

Folder	Purpose / Key Components
/Models/	Entity Framework Core entities representing database tables: ApiKey, AuditLog, TransactionAttempt, ExternalIdLink, Invoice, Customer, OAuthToken, IdempotencyRecord, etc.
/Repositories/	Data access repositories implementing CRUD logic and query helpers (Repository.cs, PaginatedResult, etc.).
/Migrations/	EF Core migrations and snapshot files for database evolution.
/Extensions/	Query helpers and LINQ extensions.
Root files	ApplicationContext and ApplicationContextFactory initialise the DB context and seed initial ApiKeys or config records.

Cross-Project Dependencies

- **API → Services:** Controllers invoke interfaces from Sage200Microservice.Services.
- **Services → Data:** Service implementations depend on repositories and entities from Sage200Microservice.Data.
- **Data → API:** None – ensures clean separation and dependency inversion.

4. API Access & Validation

All requests into the **Sage200APIMicroservice** — whether received through **HTTP endpoints** or via **Kafka inbound topics** — must carry a valid and traceable authorisation context.

This ensures tenant isolation, secure data flow, and full audit traceability across all stages of processing.

Required Request Metadata

Header / Field	Required	Purpose / Behaviour
X-Site	Optional	Specifies the Sage Site Name . If omitted, defaults to the configured value in appsettings.json: Sage:DefaultSite.
X-Company	Optional	Specifies the Sage Company Identifier . If omitted, defaults to the configured value in appsettings.json: Sage:DefaultCompanyId.
X-API-Key	✔ Required	Must match a valid record in the ApiKeys table (ApiKeys.Key). The resolved key determines tenant scope and permissions.
ExternalRef	✔ Required	Unique external identifier for the record or transaction within the calling application (e.g., Customer GUID, Transaction URN). Used to link with ExternalIdLinks.ExternalRef and associate with ApiKeys.Id.
Idempotency-Key	✔ Required (for CREATE ops)	Used to detect duplicate submissions and ensure idempotent operations. Hash stored in IdempotencyRecords for replay safety.
Correlation-Id	Auto-generated	Injected by middleware if not supplied. Used to correlate logs, Kafka messages, and Sage API calls across the pipeline.

Validation Logic (Unified Across HTTP & Kafka)

Validation Step	Description / Action
1 Key Verification	The X-API-Key is looked up in ApiKeys where IsActive = 1 and ExpiresAt > UTC_NOW().
2 Context Enrichment	If X-Site or X-Company are omitted, defaults are applied from configuration.
3 ExternalRef Binding	Each inbound request binds (ApiKeyId, ExternalRef) pair to identify or create the correct ExternalIdLinks record.
4 Idempotency Check	If an identical Idempotency-Key hash exists for the same (ApiKeyId, EntityType), the request is treated as a duplicate and the previous response is replayed without re-posting to Sage.
5 Audit Logging	Every request (valid or invalid) is persisted to the AuditLog table, capturing: Timestamp, EventType, Status, CorrelationId, ClientId, ExternalRef, and Description.
6 Rejection Handling	Invalid keys or expired tokens result in HTTP 401 Unauthorized or Kafka DLQ publication, but are still logged in the database for traceability.

Kafka Message Header Equivalents

Kafka messages published from calling applications (e.g., CymBuild) must include the same context as HTTP requests.

The microservice's consumers automatically read these values from Kafka message headers.

Kafka Header	Equivalent HTTP Header	Required	Notes
site_name	X-Site	Optional	Defaults to appsettings configuration if not provided.
company_id	X-Company	Optional	Used to target a specific Sage company database.
api_key	X-API-Key	✔ Required	Mapped to ApiKeys.Key.
external_ref	ExternalRef	✔ Required	Calling application's unique identifier for the entity.
idempotency_key	Idempotency-Key	✔ Required for create operations.	
correlation_id	Correlation-Id	Auto	Generated if not provided.

Rejection Workflow

When validation fails, the request is rejected but **never silently dropped**:

Scenario	HTTP Outcome	Kafka Outcome	Audit Behaviour
Invalid API Key	401 Unauthorized	Message published to *_DLQ with reason="Invalid API Key"	AuditLog entry with Severity=Error, Status=Failure.
Expired API Key	401 Unauthorized	*_DLQ with reason="API Key Expired"	Same as above.
Missing ExternalRef	400 Bad Request	*_DLQ with reason="Missing External Reference"	Recorded in AuditLog with context fields null.
Duplicate Idempotency Key	200 OK (Replay)	Commit offset with "Duplicate ignored"	Logged with Severity=Info, Status=Success.

Database Entities Involved

Table	Purpose	Key Columns
ApiKeys	Stores API key credentials and expiry.	Id, Key, IsActive, ExpiresAt
ExternalIdLinks	Maps external calling-app references to Sage URNs/IDs.	AppId, ExternalRef, SageUrn, SageId
TransactionAttempts	Tracks each inbound request or message processing attempt.	CorrelationId, ApiKeyId, ProcessingStatus
IdempotencyRecords	Maintains processed request hashes to prevent duplication.	KeyHash, Resource, RequestHash
AuditLog	Central log for all requests, including rejections and successes.	Timestamp, EventType, Severity, Status

Security & Observability Notes

- **Zero-trust assumption:** every inbound message or API call must prove identity through ApiKeys.Key.
- **Central logging:** all activity, even failed attempts, is captured with CorrelationId to enable forensic traceability.
- **Replay protection:** duplicate idempotent keys instantly short-circuit execution to avoid duplicate Sage transactions.
- **Default tenancy:** If SiteName or CompanyID are omitted, the microservice defaults to its primary tenant defined in configuration.

5. Kafka Topics & Message Workflow

This section defines how messages move between the **calling applications** (e.g. CymBuild), **Kafka**, the **Sage200APIMicroservice**, and **Sage 200**, including payload formats, header requirements, and system behaviours.

All topics use **underscore naming** for consistency.

5.1 Topic Catalogue

Inbound (requests → Sage200APIMicroservice)

Topic	Purpose	Producer	Consumer (Group)
MDM_INVOICE	Create or update a Customer → Create SOP Order → Generate Sales Invoice.	Calling applications (e.g. CymBuild)	sage200microservice_requests
MDM_CUSTOMER	Create or update Customer records only.	Calling applications	sage200microservice_requests
MDM_SOP_ORDER (optional)	Create SOP Order only (no automatic invoicing).	Calling applications	sage200microservice_requests

For MVP, the system will process **MDM_INVOICE** as the primary inbound topic.

Messages are keyed by correlation_id (preferred) or idempotency_key for partitioning and replay safety.

Outbound (results ← Sage200APIMicroservice)

Topic	Purpose	Producer	Consumer (Example)
MDM_INVOICE_RESULTS	Outcome of MDM_INVOICE orchestration.	Sage200APIMicroservice	CymBuild
MDM_CUSTOMER_RESULTS	Outcome of MDM_CUSTOMER processing.	Sage200APIMicroservice	CymBuild
MDM_SOP_RESULTS	Outcome of MDM_SOP_ORDER processing.	Sage200APIMicroservice	CymBuild
*_DLQ	Dead-letter queue for permanent failures.	Sage200APIMicroservice	Operations / monitoring systems

5.2 Required Kafka Headers

Header	Required	Purpose
api_key	✓	The calling application's API key (ApiKeys.Key). Used to validate access and resolve ApiKeys.Id.

Header	Required	Purpose
external_ref	✓	The calling application's unique identifier for the entity or transaction (string or GUID). Stored in ExternalIdLinks.ExternalRef.
site_name	Optional	Sage site name. Defaults to appsettings.json:Sage:DefaultSite if not supplied.
company_id	Optional	Sage company identifier. Defaults to appsettings.json:Sage:DefaultCompanyId if not supplied.
idempotency_key	✓ (for create ops)	Used to ensure unique request processing. Stored hashed (SHA-512 Base64) for duplicate prevention.
correlation_id	Optional	Used for tracing and correlation. Auto-generated if missing.

Messages missing required headers are routed to the appropriate *_DLQ topic and logged in the internal AuditLog.

5.3 Inbound Payload Schemas

Each message contains a structured JSON payload defining the business entity.

For the MVP, **MDM_INVOICE** is the orchestration entry point, combining **Customer**, **SOP Order**, and **Invoice** data.

5.3.1 MDM_INVOICE (MVP Orchestrator)

Intent:

To upsert the Customer, create an SOP Order with one or more **non-stock service lines**, and generate a Sales Invoice in Sage.

JSON Schema (Revised):

```
{
  "customer": {
    "customer_code": "string (optional, used for lookup)",
    "sage_code": "string (optional direct Sage code)",
    "name": "string",
    "contacts": [
      {
        "first_name": "string",
        "last_name": "string",
        "emails": [ { "email": "string", "is_default": true } ],
        "telephones": [ { "number": "string", "is_default": true } ]
      }
    ],
    "main_address": {
      "address_line_1": "string",
      "address_line_2": "string",

```

```

    "city": "string",
    "postcode": "string"
  }
},
"sop_order": {
  "document_no": "string (required; e.g. invoice number)",
  "customer_reference": "string (optional)",
  "currency": "string (optional; default 'GBP')",
  "lines": [
    {
      "nominal_code": "string (optional; default 31010)",
      "description": "string (required)",
      "quantity": 1,
      "unit_price": 0.00,
      "tax_code": "string (optional; default 10 = PLSTD)",
      "currency": "string (optional; default GBP)"
    }
  ]
},
"invoice": {
  "document_date": "YYYY-MM-DD",
  "reference": "string (usually same as document_no)",
  "notes": "string (optional)"
}
}

```

Example:

```

{
  "customer": {
    "name": "Star Builders Ltd",
    "contacts": [
      {
        "first_name": "Sam",
        "last_name": "Jones",
        "emails": [{ "email": "sam.jones@starbuilders.com", "is_default": true }]
      }
    ],
    "main_address": {
      "address_line_1": "42 High Street",
      "city": "Bristol",
      "postcode": "BS1 5AA"
    }
  },
  "sop_order": {
    "document_no": "INV-2025-000123",
    "currency": "GBP",
    "lines": [
      { "nominal_code": "31010", "description": "Consultancy Service", "quantity": 1, "unit_price": 200.0,
"tax_code": "10" }
    ]
  },
  "invoice": {

```

```
"document_date": "2025-10-15",
"reference": "INV-2025-000123",
"notes": "Invoice for October consultancy"
}
}
```

5.3.2 MDM_CUSTOMER (Upsert Only)

Used to create or update customers directly, without generating SOP or invoice documents.

```
{
  "customer": {
    "customer_code": "string (optional)",
    "name": "string",
    "contacts": [ ... ],
    "main_address": { ... }
  }
}
```

5.3.3 MDM_SOP_ORDER (Optional – Create Only)

Creates a Sales Order (SOP) entry directly without generating an invoice.

```
{
  "sop_order": {
    "document_no": "string",
    "customer_code": "string (optional if customer provided)",
    "currency": "string (default GBP)",
    "lines": [ ... ]
  },
  "customer": { "... optional upsert if customer_code not provided ..." }
}
```

5.4 Customer Account Code Generation Logic

When the inbound customer block does **not** specify a `sage_code` and no mapping exists in `ExternalIdLinks`:

1. Extract the **first three alphabetic characters** of the customer's name (name field).
 - Example: *Star Builders Ltd* → STA
2. Generate a **three-digit sequence** starting from 001.
 - Check STA001, STA002, etc., until a unique code is found.
3. Confirm uniqueness via:
 - `ExternalIdLinks` (local database), and
 - Sage API (GET `/customers?code=STA001`).
4. Once confirmed, store the code in:
 - `ExternalIdLinks.ExternalRef` for lookup, and

- Sage as the permanent account code.

5. Codes are uppercase, immutable once created.

Format: AAA###

Example Results:

- *Star Builders Ltd* → STA001
- *Smith Engineering Co* → SMI001
- *Alpha Systems* → ALP001

This is implemented in `CustomerService.GenerateAccountCodeAsync()` and called during `UpsertCustomerAsync()`.

5.5 Outbound Result Envelope

All outbound results (success or failure) use a consistent envelope (`ResultMessageEnvelope`):

```
{
  "correlationId": "string",
  "idempotencyKey": "string",
  "externalRef": "string",
  "apiKeyId": 123,
  "entityType": "SalesInvoice|Customer|SopOrder",
  "status": "Success|Failure",
  "sageUrn": "string",
  "sageId": 123456,
  "errors": [{ "code": "string", "message": "string" }],
  "receivedAtUtc": "2025-10-15T12:34:56Z",
  "durationMs": 2500
}
```

5.6 Processing Flow (Inbound → Outbound)

1. **Consume Message** from inbound topic (MDM_INVOICE, MDM_CUSTOMER, etc.).
2. **Validate Headers** → `api_key`, `external_ref`, and `idempotency_key`.
3. **Resolve Context** → `ApiKeyId`, `SiteName`, `CompanyId`.
4. **Idempotency Check** → Prevent duplicate reprocessing.
5. **Customer Step:**
 - Lookup by `sage_code` or `customer_code` via `ExternalIdLinks`.
 - If missing → generate new code via `GenerateAccountCodeAsync()`.
 - Upsert via Sage API (`/customers`).
 - Store Sage URN and ID in `ExternalIdLinks`.
6. **SOP Order Step:**

- Convert non-stock lines → nominal + tax codes.
- Create order (/sales_orders).
- Capture Sage Order URN and ID.

7. Invoice Step:

- Create invoice from SOP Order (/sales_invoices).
- Capture Sage Invoice URN and ID.

8. Persist:

- TransactionAttempts for idempotent tracking.
- ExternalIdLinks mapping for future reference.
- AuditLog for all events.

9. Publish Result:

- To appropriate results topic (*_RESULTS).

10. Handle Failures:

- Publish structured DlqEnvelope to *_DLQ and commit Kafka offset.

5.7 Payment Export (Airflow Integration)

Payments and allocations occur **in Sage**.

The microservice does **not** accept Kafka payment requests.

Instead:

- **Airflow** triggers the microservice periodically (e.g., every 15 minutes) via:
 - /api/payments/export-jobs or
 - /api/sync/fetch-sage-updates
- The microservice queries Sage for **allocations or credit notes** linked to known ExternalIdLinks.ExternalRef values.
- Results are published to Kafka (MDM_PAYMENT_RESULTS) or returned via HTTP for reconciliation by the calling app.

5.8 Compliance with Sage OpenAPI

- All outbound DTOs match Sage OpenAPI definitions (sales.json, sop.json).
- nominal_code, tax_code, currency, and address structures are validated according to Sage schema.
- Missing optional fields are ignored; nulls are not serialized.

- Field mapping is verified against SAGE – Field Mapping.xlsx for release readiness.

6. HTTP Endpoints & Airflow Workflow

Although the **Sage200APIMicroservice** primarily operates asynchronously through **Kafka**, it also exposes a limited **HTTP surface** for operational control, diagnostics, and scheduled payment / allocation synchronization.

These endpoints support daily automation via **Apache Airflow**, manual administrative testing, and continuous service health checks.

6.1 Overview of API Layer

Category	Purpose	Example Endpoints	Authentication
Payments Export & Allocation Sync	Retrieve and publish payments, allocations, and credit notes from Sage 200 for invoices previously created by calling applications.	/api/payments/export-jobs, /api/payments/allocations/check	API Key headers (same validation as Kafka)
System Health & Diagnostics	Validate Kafka broker, Sage API, and database connectivity.	/health, /health/kafka, /status	Open (read-only)
OAuth Refresh	Refresh Sage API access token via OAuth 2.0 Client Credentials.	/api/auth/refresh-token	Internal use (Airflow / Background Job)
Manual Replay / Debug	Replay or inspect Kafka messages (non-production / admin-gated).	/api/kafka/replay	API Key or Internal Auth

All incoming HTTP requests carry the same **context headers** used in Kafka processing, enabling identical **authorization, auditing, and correlation**.

6.2 Shared Headers for HTTP Endpoints

Header	Required	Purpose
X-API-Key		The calling application's API key (ApiKeys.Key). Must be active and not expired.

Header	Required	Purpose
X-External-Ref		The calling app's unique reference for the entity (e.g., its invoice URN or transaction ID).
X-Site	Optional	Sage site name – defaults from appsettings.json if blank.
X-Company	Optional	Sage company ID – defaults from appsettings.json if blank.

Each call passes through the **same middleware pipeline** used for Kafka messages:

1. Validate API key (IsActive = true, ExpiresAt > UtcNow).
2. Resolve ApiKeys.Id, SiteName, and CompanyId.
3. Log structured AuditLog entry for every request (action, status, latency, caller).

6.3 Payment & Allocation Export Flow

Payments and allocations are **originated only inside Sage 200**.

The microservice acts as a **synchronization agent**, allowing external apps (like *CymBuild*) to learn when their invoices have been **partially or fully allocated**.

This uses the ExternalIdLink table to correlate a calling app's ExternalRef with Sage's SageUrn and to track allocation progress.

6.3.1 Airflow Job Trigger

Airflow Task	Target Endpoint	Method	Purpose
ExportSagePayments	/api/payments/export-jobs	GET	Identify invoices needing allocation checks (daily run).
FetchAllocationChanges	/api/payments/allocations/check	GET	Retrieve allocation updates for the caller's invoices.
PublishPaymentResults	/api/sync/fetch-sage-updates	POST	Publish results back to Kafka (MDM_PAYMENT_RESULTS).
RefreshSageToken	/api/auth/refresh-token	POST	Refresh OAuth token before Sage API access.

Airflow DAG Schedule: once per day (typically 02:00 UTC).

Execution Flow

1. Airflow refreshes Sage OAuth token.
2. Calls /api/payments/export-jobs → microservice queries ExternalIdLink for invoices where EntityType='SalesInvoice' AND (IsFullyAllocated = 0 OR NULL).

3. For each candidate, microservice queries Sage API for allocations / balances.
 4. Updates AllocatedValue, OutstandingValue, timestamps, and sets IsFullyAllocated=1 when paid.
 5. Builds a PaymentResultEnvelope or AllocationUpdate for each change.
 6. Publishes results to MDM_PAYMENT_RESULTS or returns them in the HTTP response.
 7. Writes AuditLog and TransactionAttempt records for traceability.
-

6.3.2 PaymentResultEnvelope Schema

```
{
  "correlationId": "PAY-2025-000567",
  "externalRef": "BILL-7845",
  "apiKeyId": 32,
  "entityType": "Payment",
  "status": "Success",
  "sageUrn": "urn:sage:sales_invoice:123456",
  "sageId": 4556,
  "amount": 250.00,
  "currency": "GBP",
  "method": "BACS",
  "receivedAtUtc": "2025-10-29T15:30:00Z"
}
```

This mirrors ResultMessageEnvelope but adds payment-specific fields such as amount, method, and currency.

6.4 Diagnostic Endpoints

6.4.1 /health

Basic readiness probe.

```
{
  "status": "Healthy",
  "timestamp": "2025-10-30T10:00:00Z"
}
```

6.4.2 /health/kafka

Checks Kafka broker connectivity using KafkaConsumerLivenessHealthCheck.
Returns Healthy, Degraded, or Unhealthy based on metadata.

6.4.3 /status

Aggregated runtime summary.

```
{
  "kafka": "Healthy",
  "database": "Healthy",
  "sageApi": "Healthy",
  "pendingTransactions": 2,
  "failedTransactions": 0
}
```

}

6.5 Allocation Workflow (Detail)

This expands on the internal mechanics introduced earlier.

6.5.1 Database Model and State

ExternalIdLink for SalesInvoice holds:

- AllocatedValue, OutstandingValue
- IsFullyAllocated
- LastAllocationCheckUtc, LastAllocationChangeUtc

Query for daily run:

```
SELECT *  
FROM ExternalIdLinks  
WHERE EntityType = 'SalesInvoice'  
AND (IsFullyAllocated = 0 OR IsFullyAllocated IS NULL);
```

6.5.2 Daily Endpoint (/api/payments/allocations/check)

HTTP GET endpoint paginates through candidate invoices, queries Sage for allocation data, updates DB, and returns changes.

See **Section 6.3–6.10** (AllocationUpdate model, response shape, and pseudocode).

6.5.3 Kafka Result Publishing

Optionally, each allocation change is also emitted to:

- MDM_INVOICE_RESULTS
- MDM_INVOICE_RESULTS_DLQ (on failure)

Payload includes: externalRef, apiKeyId, sageUrn, allocatedValue, outstandingValue, and isFullyAllocated.

6.6 Error Handling & Logging

Error Type	HTTP Code	Action
Invalid / Expired API Key	401	Log AuditLog (Unauthorized); stop processing.
Missing Header	400	Log AuditLog (Bad Request).
Sage API Failure	502	Record TransactionAttempt (status = Failure); retry next run.
Unexpected Exception	500	Log AuditLog (Error + stack); trigger alert.

Every HTTP call generates:

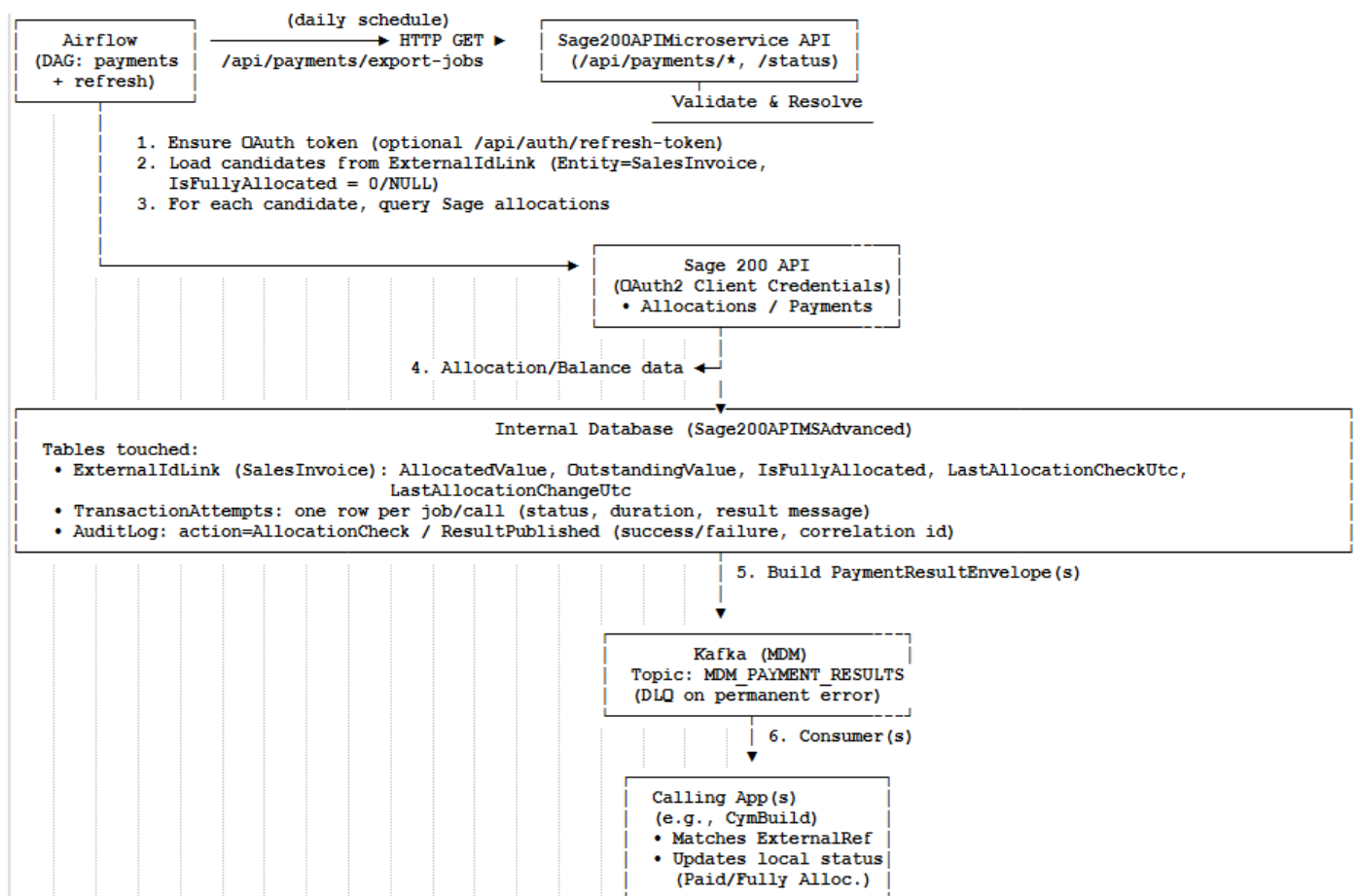
- AuditLog entry (Category = System / Business, Action = API or AllocationCheck, Status = Success | Failure).

- TransactionAttempt record for end-to-end traceability.

6.7 Sage OAuth Integration (Quick Overview)

- Uses **OAuth 2.0 Client Credentials**.
- Token cached securely and refreshed by:
 - background service on expiry, or
 - Airflow `/api/auth/refresh-token` call.
- Credentials encrypted in `appsettings.json` or KeyVault.
- Failures to refresh cause `/status` to report `sageApi=Unhealthy`.

6.8 Endpoint Workflow Diagram



6.9 Audit & Metrics Summary

Metric	Description
allocations_candidates_total	Number of invoices checked per run

Metric	Description
allocations_changed_total	Invoices with allocation value changes
allocations_fully_allocated_total	Newly paid invoices
allocations_api_failures_total	Sage API errors
allocations_check_duration_ms	Duration of allocation check operation

AuditLog sample:

2025-10-31 02:10Z | ApiKeyId=42 | Action=AllocationCheck |
Resource=SalesInvoice | Checked=112 | Updated=9 |
FullyAllocated=4 | Status=Success

Summary of Section 6

- **HTTP endpoints** complement Kafka for synchronous control, diagnostics, and daily payment/allocations.
- **Airflow DAG** runs once per day to update allocations and publish results.
- **Shared header model** provides unified authentication + auditing for HTTP and Kafka.
- **ExternalIdLink enhancements** make allocation tracking efficient and idempotent.
- **Audit & metrics** ensure observability and compliance.
- The design cleanly separates concerns:
 - **Sage200APIMicroservice** → sync agent.
 - **Sage 200 API** → system of record.
 - **CymBuild / other apps** → consumer of results.

7. Kafka Contracts & Orchestrations

The **Kafka integration layer** provides asynchronous, durable communication between multiple calling applications (CymBuild and others) and the **Sage200APIMicroservice**.

Each topic is entity-specific and uses a consistent message envelope to maintain traceability, idempotency, and auditability across systems.

7.1 Topic Naming Convention (underscore style)

Topic Name	Direction	Entity / Purpose	Description
MDM_CUSTOMER	Inbound → Microservice	Customer creation / update	Upsert customer records into Sage 200.
MDM_SOP_ORDER	Inbound → Microservice	Sales Order processing	Create non-stock SOP orders in Sage 200.
MDM_INVOICE	Inbound → Microservice	Invoice generation from SOP order	Creates invoice linked to SOP order and customer.
MDM_CUSTOMER_RESULTS	Outbound ← Microservice	Customer ack/result	Response payload after Sage processing.
MDM_SOP_RESULTS	Outbound ← Microservice	SOP order ack/result	Sage 200 SOP confirmation or error.
MDM_INVOICE_RESULTS	Outbound ← Microservice	Invoice ack/result	Invoice creation status and Sage URN.
MDM_PAYMENT_RESULTS	Outbound ← Microservice	Payment allocation results	Published by daily Airflow job.
MDM_DLQ	Outbound ← Microservice	Dead Letter Queue	For messages that cannot be processed.

All topics are **partition-keyed by correlationId** for deterministic ordering and traceability.

7.2 Common Envelope Structure (for Inbound Messages)

Each Kafka message wraps a consistent JSON envelope:

```
{  
  "correlationId": "INV-2025-0001",  
  "idempotencyKey": "2a7c74c3-498e-4b8d-8a13-9c55b0bb91f6",  
  "siteName": "MainSite",  
  "companyId": "SAGE200UK",  
  "apiKey": "f3927b41-d32a-4af8-bc5b-0a98fdb90a12",  
  "externalRef": "CUST-7845",  
  "timestampUtc": "2025-10-31T08:40:00Z",  
  "entity": { /* entity-specific payload below */ }  
}
```

Field	Required	Description
correlationId	✓	Unique trace ID for message chain (used for partition key).
idempotencyKey	✓	Deterministic token used to prevent duplicate writes (replayed safely).

Field	Required	Description
siteName	Optional	Overrides default Sage site in appsettings.json.
companyId	Optional	Overrides default Sage company.
apiKey	✓	Calling app's key (looked up in ApiKeys table).
externalRef	✓	Calling app's local URN or record identifier.
entity	✓	Business payload (Customer, SOP Order, Invoice, etc.).

Validation rules mirror HTTP headers (see Section 4).

7.3 Entity-Specific Schemas

7.3.1 Customer (MDM_CUSTOMER)

```
{
  "customerCode": "STA001",
  "customerName": "Station Services Ltd",
  "email": "accounts@station.co.uk",
  "telephone": "+44 208 555 2365",
  "address": {
    "line1": "24 High Street",
    "line2": "",
    "city": "London",
    "postcode": "SW1A 1AA"
  },
  "currency": "GBP",
  "taxCode": "10"
}
```

- If customerCode missing, microservice generates unique code AAA000 format.
- taxCode defaults to 10 (PLSTD) if not supplied.
- Upsert logic: If ExternalIdLink exists for externalRef → Update; else create and insert mapping.

7.3.2 SOP Order (MDM_SOP_ORDER)

```
{
  "orderNumber": "SO-001004",
  "customerCode": "STA001",
  "currency": "GBP",
  "lines": [
    {
      "description": "On-site service call – 2 hours",
      "nominalCode": "31010",
      "quantity": 2,
      "unitPrice": 60.00,
      "taxCode": "10"
    }
  ]
}
```

```
}  
]  
}
```

- All lines are **non-stock**.
- If nominalCode absent → use default 31010.
- If taxCode absent → default 10.
- Produces a Sage 200 Sales Order; Sage URN and Order Number persisted to ExternalIdLink.

7.3.3 Invoice (MDM_INVOICE)

```
{  
  "invoiceNumber": "INV-001004",  
  "sopOrderNumber": "SO-001004",  
  "currency": "GBP",  
  "lines": [  
    {  
      "description": "On-site service call – 2 hours",  
      "nominalCode": "31010",  
      "unitPrice": 60.00,  
      "quantity": 2,  
      "taxCode": "10"  
    }  
  ]  
}
```

- Tightly coupled with the preceding SOP Order.
- When created, the microservice will:
 1. Resolve Customer via ExternalIdLink.
 2. Create Invoice via Sage API.
 3. Insert/Update mapping to Sage URN in ExternalIdLink.
 4. Publish result to MDM_INVOICE_RESULTS.

7.4 Outbound Result Contracts

Each consumer publishes a standardized ResultMessageEnvelope (see Stage 11 implementation):

```
{  
  "correlationId": "INV-2025-0001",  
  "externalRef": "CUST-7845",  
  "apiKeyId": 12,  
  "entityType": "SalesInvoice",  
  "status": "Success",  
  "sageUrn": "urn:sage:invoice:12039",  
  "sageId": 12039,  
  "receivedAtUtc": "2025-10-31T08:44:12Z",  
  "durationMs": 942,  
}
```



```
"errors": null
}
```

- **Success path:** updates TransactionAttempts, ExternalIdLink, and AuditLog.
 - **Failure path:** publishes to MDM_DLQ and commits offset.
-

7.5 Idempotency & Correlation Rules

Field	Purpose	Behaviour
idempotencyKey	Deduplicate inbound Kafka events.	Hash stored in TransactionAttempts.IdempotencyKeyHash; rejected if already seen.
correlationId	Trace entire transaction across Kafka, DB, and logs.	Used as partition key and AuditLog identifier.
externalRef	Map caller's record to Sage record.	Stored in ExternalIdLink.ExternalRef.

Every Kafka operation is atomic with respect to its TransactionAttempt record for exactly-once semantics.

7.6 Consumer Behaviour & DLQ Policy

- **Inbound consumers:** InvoiceRequestConsumer, CustomerRequestConsumer, SopOrderConsumer.
- **Outbound result consumers:** InvoiceResultConsumer, CustomerResultConsumer, SopResultConsumer.
- **DLQ Handling:** Any exception during processing → serialize DlqEnvelope to MDM_DLQ.
- Each DLQ message includes:
 - correlationId
 - reason
 - originalPayload (UTF-8, trimmed)
 - occurredUtc timestamp

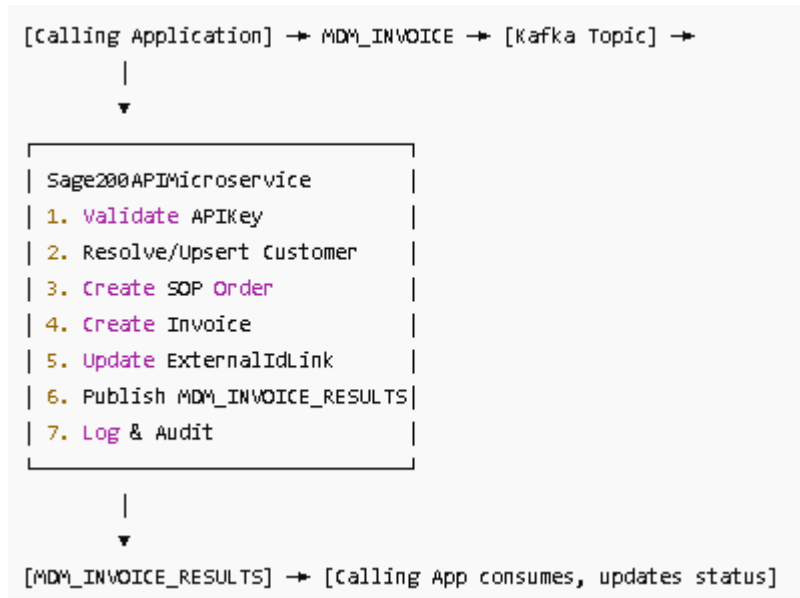
This ensures audit completeness and replay support.

7.7 Versioning & Schema Compatibility

- **Version Field:** Optional schemaVersion may be included at root level. Default = 1.0.

- **Backward Compatibility:** New fields are always optional and ignored by older consumers.
 - **Breaking changes:** require new topic suffix, e.g., MDM_INVOICE_V2.
 - **Schema Validation:** Performed by the Kafka consumer layer before passing to orchestrator. Invalid payloads route directly to DLQ.
-

7.8 Orchestration Flow Diagram



Summary of Section 7

- Kafka topics use underscore naming and consistent schemas.
 - All messages carry a standard envelope with trace IDs, idempotency, and API context.
 - Entity payloads follow Sage OpenAPI contracts (extended for non-stock rules and defaults).
 - Outbound results and DLQ use canonical envelopes for resilient processing.
 - Orchestration ensures idempotent, traceable end-to-end flows from CymBuild → Kafka → Sage → Kafka → CymBuild.
-

7A. Kafka Schema References (Developer Appendix)

These schema definitions describe the **expected JSON payloads** for all inbound Kafka topics consumed by the **Sage200APIMicroservice**.

They are written in **OpenAPI v3.1 syntax** and can be imported directly into tools like **Swagger**, **Postman**, or **Insomnia** for schema validation.

7A.1 Common Envelope Schema

Envelope:

type: object

required:

- correlationId
- idempotencyKey
- apiKey
- externalRef
- entity

properties:

correlationId:

type: string

example: "INV-2025-0001"

description: Unique message chain identifier (used for partition key and audit correlation).

idempotencyKey:

type: string

format: uuid

example: "2a7c74c3-498e-4b8d-8a13-9c55b0bb91f6"

siteName:

type: string

nullable: true

example: "MainSite"

description: Optional Sage site override; defaults from appsettings.json.

companyId:

type: string

nullable: true

example: "SAGE200UK"

description: Optional Sage company override.

apiKey:

type: string

example: "f3927b41-d32a-4af8-bc5b-0a98fdb90a12"

description: Calling application's key (looked up in ApiKeys table).

externalRef:

type: string

example: "CUST-7845"

description: Calling application's unique record or transaction reference.

timestampUtc:

type: string

format: date-time

example: "2025-10-31T08:40:00Z"

schemaVersion:

type: string

example: "1.0"

entity:

oneOf:

- \$ref: '#/components/schemas/CustomerPayload'
 - \$ref: '#/components/schemas/SopOrderPayload'
 - \$ref: '#/components/schemas/InvoicePayload'
-

7A.2 Customer Payload (MDM_CUSTOMER)

CustomerPayload:

type: object

required:

- customerName

properties:

customerCode:

type: string

example: "STA001"

description: Sage customer code (if omitted, system will auto-generate).

customerName:

type: string

example: "Station Services Ltd"

email:

type: string

format: email

example: "accounts@station.co.uk"

telephone:

type: string

example: "+44 208 555 2365"

address:

type: object

properties:

line1:

type: string

example: "24 High Street"

line2:

type: string

example: ""

city:

type: string

example: "London"

postcode:

type: string

example: "SW1A 1AA"

currency:

type: string

default: "GBP"

taxCode:

type: string

default: "10"

description: Default tax code (PLSTD) if not supplied.

7A.3 SOP Order Payload (MDM_SOP_ORDER)

SopOrderPayload:

type: object

required:

- customerCode

- lines

properties:

orderNumber:
 type: string
 example: "SO-001004"

customerCode:
 type: string
 example: "STA001"
 description: Link to existing customer (Sage code or ExternalIdLink).

currency:
 type: string
 default: "GBP"

lines:
 type: array
 minItems: 1
 items:
 type: object
 required:
 - description
 - quantity
 - unitPrice
 properties:
 description:
 type: string
 example: "On-site service call – 2 hours"
 nominalCode:
 type: string
 default: "31010"
 description: Default non-stock nominal sales code.
 quantity:
 type: number
 example: 2
 unitPrice:
 type: number
 format: decimal
 example: 60.00
 taxCode:
 type: string
 default: "10"
 description: Default = 10 (PLSTD).

7A.4 Invoice Payload (MDM_INVOICE)

InvoicePayload:
 type: object
 required:
 - invoiceNumber
 - sopOrderNumber
 - lines
 properties:
 invoiceNumber:
 type: string
 example: "INV-001004"
 sopOrderNumber:

type: string
example: "SO-001004"
description: Must reference an existing SOP order in Sage.
currency:
type: string
default: "GBP"
lines:
type: array
minItems: 1
items:
type: object
required:
- description
- quantity
- unitPrice
properties:
description:
type: string
example: "On-site service call – 2 hours"
nominalCode:
type: string
default: "31010"
quantity:
type: number
example: 2
unitPrice:
type: number
format: decimal
example: 60.00
taxCode:
type: string
default: "10"
description: Default = PLSTD

7A.5 Payment Result Envelope (Outbound)

PaymentResultEnvelope:

type: object
required:
- correlationId
- externalRef
- apiKeyId
- entityType
- status
- amount
properties:
correlationId:
type: string
example: "PAY-2025-000567"
externalRef:
type: string
example: "INV-7845"

apiKeyId:
type: integer
example: 32
entityType:
type: string
enum: [Payment]
status:
type: string
enum: [Success, Failure]
sageUrn:
type: string
example: "SAGEPAY-001"
sageId:
type: integer
example: 4556
amount:
type: number
example: 250.00
allocatedValue:
type: number
example: 250.00
outstandingValue:
type: number
example: 0.00
currency:
type: string
default: "GBP"
method:
type: string
example: "BACS"
receivedAtUtc:
type: string
format: date-time
example: "2025-10-29T15:30:00Z"

Developer Notes

- **Envelope-first validation:** every Kafka message must conform to Envelope, then validate its nested entity schema.
 - **Defaults applied automatically** by microservice if omitted (currency=GBP, taxCode=10, nominalCode=31010).
 - **Idempotency enforced** using idempotencyKey hash in database.
 - **Schema evolution:** all future changes must append optional fields; mandatory field changes trigger new topic suffix _V2.
-

8. Internal Database Design & Mapping Logic

This section explains how the **Sage200APIMicroservice** maintains data integrity, idempotency, and auditability through its internal SQL database (**Sage200APIMSAAdvanced**).

The database acts as the **system of record** for all transaction mappings, Kafka traceability, and external system correlation (between CymBuild and Sage 200).

8.1 Overview

The internal database ensures that every message processed through the microservice is:

- **Traceable** (end-to-end via CorrelationId and ExternalRef)
- **Idempotent** (safe to replay via IdempotencyKeyHash)
- **Auditable** (recorded in AuditLog for every attempt and outcome)
- **Externally referenceable** (linked to both the calling app and Sage entity)

At a high level:

[CymBuild] → Kafka (MDM_INVOICE)

↓

[TransactionAttempts] ← audit and state tracking

↓

[ExternalIdLink] ← maps CymBuild record ↔ Sage record

↓

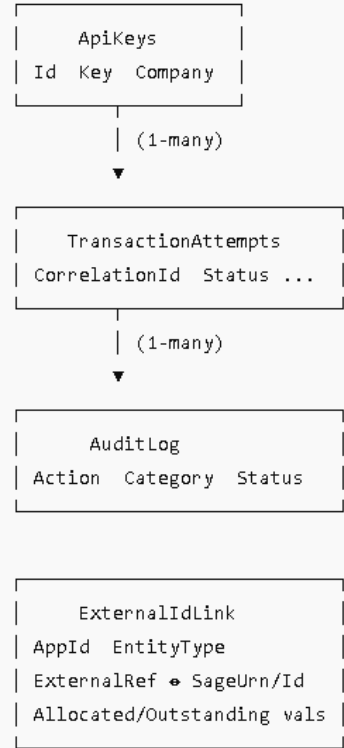
[AuditLog] ← records every action, result, and status

8.2 Core Tables

Table	Purpose	Key Fields	Relations
ApiKeys	Stores registered calling applications and authorization tokens.	Id, Key, CompanyName, IsActive, ExpiresAt	Referenced by TransactionAttempts.ApiKeyId, ExternalIdLink.AppId
TransactionAttempts	Tracks every inbound or outbound Kafka event, ensuring idempotency and replay safety.	CorrelationId, IdempotencyKeyHash, ProcessingStatus, KafkaTopic, ApiKeyId, SageUrn, SageId	Links to ApiKeys, ExternalIdLink, AuditLog
ExternalIdLink	Maps calling app entity IDs (ExternalRefs) to Sage entity	AppId, EntityType, ExternalRef, SageUrn, SageId, IsFullyAllocated,	Linked via AppId → ApiKeys.Id

Table	Purpose	Key Fields	Relations
	URNs/IDs and sync state.	AllocatedValue, OutstandingValue	
AuditLog	Captures every operation (API, Kafka, DLQ, etc.) for full traceability.	Timestamp, Action, Category, Status, Severity, CorrelationId, ApiKeyId, Description	Connected to TransactionAttempts for correlation
SageApiTokenCache <i>(optional)</i>	Stores current Sage OAuth access/refresh tokens.	AccessToken, RefreshToken, ExpiresAtUtc	Used by background refresh and Airflow
OutboxMessages <i>(future use)</i>	Reserved for transactional message-outbox pattern if required.	EventType, PayloadJson, SentAtUtc	Not yet used in MVP

8.3 Entity Relationships (Simplified Diagram)



8.4 TransactionAttempts Flow

Each Kafka or HTTP request inserts a **TransactionAttempt** record to represent its lifecycle.

Column	Purpose / Example
CorrelationId	"INV-2025-0001" – used throughout message chain
ProcessingStatus	Received, SageSuccess, SageFailure
KafkaTopic	MDM_INVOICE
IdempotencyKeyHash	SHA-512 hash of idempotencyKey
ApiKeyId	FK → ApiKeys.Id
ProcessingStartedUtc, ProcessingCompletedUtc	Performance timing
SageUrn, SageId	Sage identifiers returned by API
ResultMessage	Concise summary or error for logs
DurationMs	Measured or reported duration

Status transitions:

Received → Processing → SageSuccess | SageFailure → ResultPublished

8.5 ExternalIdLink Logic

The ExternalIdLink table is the **contract bridge** between the calling app and Sage 200:

Column	Purpose
AppId	Calling application (from ApiKeys.Id)
EntityType	Customer, SopOrder, or SalesInvoice
ExternalRef	CymBuild's unique reference for record/transaction
SageUrn, SageId	Sage 200 unique identifiers
IsFullyAllocated	1 if invoice fully paid, 0 otherwise
AllocatedValue, OutstandingValue	Used by Airflow to calculate payment states
LastAllocationCheckUtc, LastAllocationChangeUtc	Track sync timestamps

Rules:

- When OutstandingValue = 0, mark IsFullyAllocated = 1.
- Airflow daily job queries:
WHERE EntityType='SalesInvoice' AND (IsFullyAllocated=0 OR IsFullyAllocated IS NULL)
- Every payment sync updates the allocation columns and timestamps.

8.6 AuditLog Usage

Every processed event (Kafka or HTTP) results in at least one **AuditLog** entry.

This includes both system-level and business-level actions:

Field	Description
Timestamp	UTC of event
Category	Business, System, Security
Severity	Info, Warning, Error
Action	e.g., ResultReceived, PaymentSync, ApiValidation
Status	Success, Failure, InProgress
CorrelationId	Trace to TransactionAttempts
ClientId	API key or application name
Description	Detailed summary of event
DurationMs	Optional timing metric

8.7 Data Retention and Purging Strategy

Table	Retention Period	Archival Action
TransactionAttempts	6 months	Archive or summarize older rows
AuditLog	12 months	Compress into monthly aggregates
ExternalIdLink	Persistent	Never purged (business mapping)
ApiKeys	Active/inactive lifecycle	Deactivated, not deleted
SageApiTokenCache	Rolling	Auto-refreshed by job

8.8 Database Integrity Rules

1. **Foreign keys** enforce referential consistency (TransactionAttempts.ApiKeyId → ApiKeys.Id).
 2. **Unique constraints**
 - ExternalIdLink(AppId, EntityType, ExternalRef)
 - TransactionAttempts(CorrelationId)
 3. **Indexed columns**
 - CorrelationId, ExternalRef, SageUrn (for fast lookups).
 4. **Soft deletes only** if future audit archiving is needed (no hard deletes).
-

8.9 Example Data Lineage (End-to-End)

Stage	Action	Database Entry
1	CymBuild publishes MDM_INVOICE	Kafka consumer inserts TransactionAttempt
2	Customer & SOP validated/created	Upsert in ExternalIdLink
3	Invoice created in Sage	Update SageUrn/SageId in ExternalIdLink
4	Result published → MDM_INVOICE_RESULTS	TransactionAttempt → SageSuccess
5	Payment allocated (Airflow job)	Update AllocatedValue, OutstandingValue, IsFullyAllocated
6	AuditLog records all actions	Complete trace retained

Summary of Section 8

- The internal database forms the **anchor point** for all microservice activity.
 - Each entity (Customer → SOP → Invoice → Payment) links back to the same calling application context.
 - **ExternalIdLink** provides two-way mapping between CymBuild and Sage.
 - **TransactionAttempts** and **AuditLog** ensure reliable processing, diagnostics, and compliance audit.
 - Schema additions (like allocation tracking) make payment reconciliation seamless and Airflow-ready.
-

9. Service Layer & Orchestrator Responsibilities

The **service layer** defines the structured workflow that converts validated business messages into Sage 200 API operations.

At its core is the **InvoiceRequestOrchestrator**, supported by dedicated domain services such as CustomerService, SopOrderService, and SalesInvoicesService.

Each orchestrator coordinates:

1. **Data validation & lookup**
2. **Entity creation or update in Sage 200**
3. **Persistence of mappings (ExternalIdLink)**
4. **Audit and transaction updates (AuditLog, TransactionAttempts)**
5. **Result publishing back to Kafka**

9.1 Layer Overview

Layer	Purpose	Key Classes / Files
Orchestrators	Orchestrate multi-step processes (customer → order → invoice).	InvoiceRequestOrchestrator, InvoiceRequestOrchestrator
Domain Services	Direct Sage API operations per entity type.	ICustomerService, ISopOrderService, ISalesInvoicesService
Infrastructure Services	Reusable building blocks for messaging, security, OAuth, and HTTP.	IEventPublisher, OAuthClient, KafkaProducerService
Persistence Layer	EF Core context and repositories.	ApplicationContext, DbSet<T> tables
Auditing & Validation	Shared middleware and utilities.	AuditService, HeaderValidator, ApiKeyValidator

9.2 Orchestrator Role

The orchestrator is the **transactional bridge** between inbound messages and the Sage API.

Each orchestrator handles a complete **business transaction** rather than a single entity operation.

9.2.1 InvoiceRequestOrchestrator (Core MVP Example)

Inbound: MDM_INVOICE → Kafka Consumer

Steps:

1. Validate headers and API key
2. Deserialize message into InvoiceRequest (Sop + Customer context)

3. Upsert Customer via CustomerService
4. Create SOP Order via SopOrderService
5. Create Invoice via SalesInvoicesService
6. Persist TransactionAttempt and ExternalIdLink
7. Publish result → MDM_INVOICE_RESULTS
8. Record AuditLog

9.2.2 Key Responsibilities

Responsibility	Description
Validation	Verifies headers, API key, and schema compliance.
Correlation	Uses CorrelationId and IdempotencyKeyHash for deduplication.
Coordination	Calls downstream domain services in sequence with rollback safety.
Logging	Writes intermediate progress to TransactionAttempts.
Response	Publishes concise results to Kafka topic.

9.3 Domain Services

9.3.1 CustomerService

- Responsible for checking existence or creating a Sage 200 customer.
- Works against **/customers** endpoint from sales.json.

Core Flow:

1. Lookup ExternalIdLink by (Appld, ExternalRef, EntityType=Customer).
2. If found → update via Sage API (PUT).
3. If not found → create via Sage API (POST).
 - Auto-generate customer code if blank (AAA000 pattern).
4. Persist mapping (SageUrn, Sageld) in ExternalIdLink.
5. Return Sage URN to orchestrator.

9.3.2 SopOrderService

- Handles all SOP order creation and linkage to customer.
- Uses **/sales_orders** endpoint from sop.json.

Key Details:

- Lines are **non-stock**; default nominalCode = 31010, taxCode = 10.
- CustomerCode is validated before request.
- Persists mapping in ExternalIdLink (EntityType=SopOrder).

Failure Handling:

- Any Sage validation failure triggers a DLQ publish with error context.
-

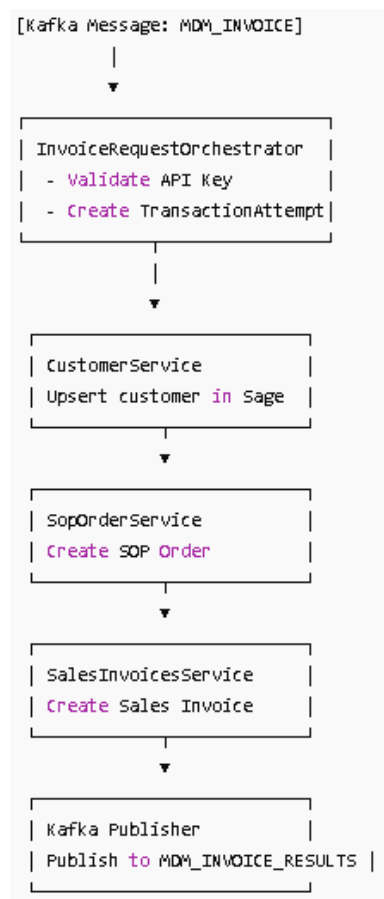
9.3.3 SalesInvoicesService

- Final step in chain: creates Sage invoice from SOP order.
- Uses **/sales_invoices** endpoint (sales.json).
- Validates SOP linkage (SOP Order must exist in ExternalIdLink).

Flow:

1. Confirm customer and SOP existence.
 2. Construct invoice model with correct Sage references.
 3. POST to Sage API /sales_invoices.
 4. On success → update ExternalIdLink (EntityType=SalesInvoice).
 5. On error → record in TransactionAttempt and DLQ if unrecoverable.
-

9.4 Service Interaction Diagram (Invoice Flow)



9.5 Database Integration (per orchestration)

Action	Table Updated	Description
Receive inbound message	TransactionAttempts	Create row with Received status
API Key validated	AuditLog	"APIKeyValidation Success"
Customer upserted	ExternalIdLink	EntityType=Customer
SOP order created	ExternalIdLink	EntityType=SopOrder
Invoice created	ExternalIdLink	EntityType=SalesInvoice
Result published	TransactionAttempts	Status → SageSuccess
Error or DLQ	TransactionAttempts / AuditLog	Failure recorded

9.6 Resilience & Retry

- **Transient faults** (timeouts, throttles, 5xx) → retried up to 3 times with backoff.
- **Permanent faults** (validation, mapping) → publish DLQ envelope immediately.
- **Manual replay endpoint** /api/kafka/replay can reprocess a given CorrelationId.

- **Idempotency** ensures that replays never duplicate customer, SOP, or invoice creation.

9.7 Sage API Integration Rules

- Each service uses a shared **OAuthClient** that manages tokens transparently.
- OAuth tokens retrieved from local cache (SageApiTokenCache) or refreshed automatically.
- All outbound HTTP requests include:
 - Authorization: Bearer {token}
 - X-Site and X-Company headers (defaults from appsettings).
- Failures are parsed and simplified before being logged in ResultMessage.

9.8 Orchestrator Error Strategy

Error Type	Response / Action	Next Step
Invalid API Key	Log AuditLog → “Unauthorized”	Stop processing
Missing ExternalRef	Log to AuditLog; DLQ publish	Stop processing
Customer not found & cannot create	DLQ	Continue other messages
Sage API timeout	Retry (3x exponential)	DLQ if still failing
Invoice creation failed	Partial success recorded	Publish Failure result

9.9 Example Orchestrator Pseudocode

```
public async Task ProcessInvoiceRequestAsync(SopOrderCreate request, HttpContext context,
Cancellation token ct)
{
    var apiKey = await _apiKeyValidator.ValidateAsync(context);
    var transaction = await _db.CreateTransactionAttemptAsync(request, apiKey);

    try
    {
        var customerUrn = await _customerService.UpsertCustomerAsync(request.Customer, apiKey, ct);
        var sopUrn = await _sopService.CreateSopOrderAsync(request, customerUrn, apiKey, ct);
        var invoiceUrn = await _invoiceService.CreateInvoiceFromSopAsync(request, sopUrn, apiKey, ct);

        await _publisher.PublishResultAsync("MDM_INVOICE_RESULTS", new { correlationId =
transaction.CorrelationId, status = "Success" });
        await _db.MarkAttemptSuccess(transaction, invoiceUrn);
    }
    catch (Exception ex)
    {
    }
```

```
    await _db.MarkAttemptFailure(transaction, ex.Message);
    await _publisher.PublishDlqAsync(transaction.CorrelationId, ex.Message);
}
}
```

9.10 Benefits of the Orchestration Model

- ✔ Enforces **data integrity** across multi-step workflows
 - ✔ Provides **atomic transaction visibility** in the database
 - ✔ Supports **replayable, idempotent** design
 - ✔ Easily extendable for future entity types (Payments, Stock, Purchase Orders)
 - ✔ Centralized auditing and logging
-

Summary of Section 9

- Orchestrators provide a **business transaction workflow**, not just data mapping.
 - Domain services encapsulate Sage 200 API logic and validation.
 - Database integration (Section 8) supports traceability, retries, and mapping.
 - Resilience, OAuth handling, and DLQ policy ensure fault tolerance.
 - The architecture scales horizontally for additional entity pipelines without major refactoring.
-

10. Error Handling, Dead-Letter Queue (DLQ), and Retry Policies

This section defines the **resilience strategy** of the Sage200APIMicroservice.

It ensures that every message—whether received via Kafka or HTTP—is processed *exactly once* where possible, retried safely on transient failures, and permanently recorded on unrecoverable errors.

10.1 Error-Handling Philosophy

Objective	Implementation
Reliability	Never lose a message. All failures must be persisted and auditable.
Idempotency	Re-processing the same message should not create duplicates.
Transparency	Every outcome—success, retry, or failure—is visible in AuditLog and TransactionAttempts.
Autonomy	Kafka consumers manage their own retry and DLQ logic without central coordination.

10.2 Error Classification

Type	Description	Retry?	DLQ?
Transient	Network blips, 5xx from Sage API, token expiry, Kafka timeout.	✓ Up to 3 attempts with exponential back-off.	✗
Permanent (Business)	Validation failure, missing mapping, duplicate idempotency key.	✗	✓
Infrastructure	DB unavailable, serialization error, config issue.	✓ (if recoverable)	✓ (if persistent > threshold)
Unknown / Exception	Unhandled runtime error.	✗	✓

10.3 Retry Policy

10.3.1 Exponential Back-Off

Attempt	Delay (± jitter)
1	10 seconds
2	30 seconds
3	2 minutes

After 3 failed attempts → message is marked **Failed** in TransactionAttempts and pushed to the **DLQ**.

10.3.2 Implementation

- Built on Polly or internal back-off logic.
- Each retry updates TransactionAttempts.RetryCount and DurationMs.
- If the final attempt fails, ProcessingStatus = "Failed" and ResultMessage holds the error reason.

10.4 Dead-Letter Queue (DLQ)

10.4.1 Purpose

Stores messages that cannot be processed after maximum retry attempts or that fail validation.

10.4.2 DLQ Topics

Entity	DLQ Topic
Invoices	MDM_INVOICE_DLQ

Entity	DLQ Topic
Customers	MDM_CUSTOMER_DLQ
SOP Orders	MDM_SOP_ORDER_DLQ
Payments (allocations)	MDM_PAYMENT_DLQ

10.4.3 DLQ Envelope Structure

```
{
  "correlationId": "INV-2025-0007",
  "entityType": "SalesInvoice",
  "topic": "MDM_INVOICE",
  "apiKeyId": 32,
  "externalRef": "INV-1023",
  "status": "Failed",
  "errorCategory": "Validation",
  "errorMessage": "Customer not found and creation failed.",
  "originalPayload": { /* full body */ },
  "timestampUtc": "2025-10-31T09:12:00Z"
}
```

10.4.4 Processing Policy

- DLQ messages remain indefinitely until manually replayed or reviewed.
- `/api/kafka/replay` endpoint allows admins to resend selected DLQ messages.
- Replay uses the same idempotency-safe flow (Section 9.6).

10.5 Audit and Visibility Integration

Every failure—transient or permanent—generates:

1. AuditLog entry

- Category = System
- Severity = Error
- Status = Failure
- Description = {ExceptionMessage}

2. TransactionAttempts update

- ProcessingStatus = Failed
- RetryCount = n
- ResultMessage = errorSummary

3. Optional DLQ Publish (for unrecoverables).

All errors are therefore queryable by CorrelationId or ExternalRef.

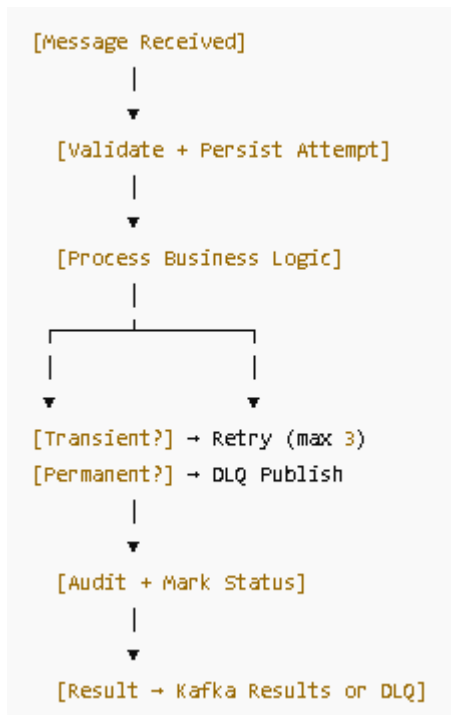
10.6 Typical Error Scenarios

Scenario	Handling	Outcome
Sage API 503 Service Unavailable	Retry × 3 → back-off	Success or DLQ
OAuth token expired	Refresh token → retry	Success
Invalid API key (header)	Reject 401, log AuditLog	No retry
Customer creation fails (validation)	DLQ publish	Recorded as Failure
Serialization error in Kafka payload	DLQ publish	Manual fix
Database timeout	Retry × 2	Success or Failure log

10.7 Logging & Monitoring

- Structured logging via Serilog → JSON logs with fields:
 - CorrelationId, ExternalRef, EntityType, Status, DurationMs, RetryCount.
 - Exported to Application Insights / Elastic Stack for central monitoring.
 - Prometheus metrics:
 - microservice_message_failures_total{entity}
 - microservice_dlq_messages_total{entity}
 - microservice_retries_total{entity}
-

10.8 Error Workflow Diagram (Conceptual)



10.9 Developer Guidelines

- ✓ Wrap all external calls in try/catch and classify exceptions.
- ✓ Never swallow exceptions—record them to AuditLog.
- ✓ Use CorrelationId on every log entry.
- ✓ Publish DLQ messages as JSON with the original payload and error context.
- ✓ Respect idempotency on retries (no duplicates).
- ✓ Manually replay DLQ only after root cause analysis.

Summary of Section 10

- **Retries** handle temporary faults automatically.
- **DLQ** captures non-recoverable failures for manual replay.
- **AuditLog + TransactionAttempts** ensure traceable, idempotent operations.
- **Monitoring hooks** feed real-time visibility to Ops and Dev teams.
- The microservice never loses a message—every failure path is recorded, classified, and recoverable.

11. Security, API Key Management & OAuth Governance

The Sage200APIMicroservice enforces a multi-layered security model designed for **distributed, multi-tenant integration** environments.

Its goal is to ensure that:

1. Only **authorized calling applications** can interact with the service.
2. All access to **Sage 200 endpoints** uses **secure, OAuth-managed tokens**.
3. All actions remain **fully auditable and traceable** in the internal database.

11.1 Security Architecture Overview

Layer	Purpose	Technology / Mechanism
Application Identity	Identifies the calling application via unique API Key.	ApiKeys table, validated by middleware.
Authorization Context	Determines if the key is active, valid, and not expired.	ApiKeys.IsActive, ApiKeys.ExpiresAt
Transport Security	Ensures encrypted traffic between systems.	HTTPS + TLS 1.2+
OAuth (Sage Integration)	Authenticates against Sage 200 API endpoints.	OAuth 2.0 Client Credentials grant
Auditing Layer	Captures security-related actions and validation failures.	AuditLog table

11.2 API Key Lifecycle

API Keys represent the identity of **calling applications** (e.g., CymBuild, other client systems). Each key links to a row in the ApiKeys table and drives all message ownership, validation, and authorization logic.

11.2.1 Table Structure: ApiKeys

Column	Description
Id	Primary key, internal reference.
Key	The API Key string (unique, securely generated GUID).
CompanyName	Name of the calling application or organization.
CreatedAt	Creation timestamp.
IsActive	Indicates if key is currently enabled.
ExpiresAt	Expiration timestamp for automatic invalidation.
ContactEmail	Used for operational alerts.

Column	Description
Permissions (<i>optional</i>)	Comma-separated scopes (future enhancement).

11.2.2 Validation Process

Each incoming HTTP request or Kafka message must include a valid X-API-Key header.

Validation steps:

1. **Lookup** in ApiKeys table by Key.
2. **Check:**
 - IsActive = true
 - ExpiresAt > UTC_NOW()
3. **If invalid:**
 - Reject with 401 Unauthorized.
 - Log entry in AuditLog with Category = Security, Severity = Error.
4. **If valid:**
 - The ApiKeys.Id value is bound to the current request context for logging and mapping.

All downstream references (in ExternalIdLink, TransactionAttempts) use this **numeric AppId** rather than the raw key.

11.2.3 Key Rotation

API Keys can be rotated safely without losing link integrity:

1. Generate new key → insert into ApiKeys table.
2. Mark old key as inactive (IsActive = false).
3. Because all mapping uses the Id FK, historical transactions remain valid.

11.3 Header-Based Authorization Context

Every HTTP or Kafka-originating request must include the **authorization context**, allowing multi-tenant separation and default fallback behavior.

Header	Required?	Purpose	Default Behavior
X-API-Key		Identifies the calling app (maps to ApiKeys.Key).	Rejects if missing or invalid.
X-Site	Optional	Sage site identifier.	Defaults to appsettings.json.

Header	Required?	Purpose	Default Behavior
X-Company	Optional	Sage company identifier.	Defaults to appsettings.json.
X-External-Ref		Calling app's record or transaction ID.	Required for linkage to ExternalIdLink.

When any header is missing or invalid, the request is rejected but **still logged** to AuditLog for traceability.

11.4 Sage OAuth 2.0 Token Governance

All direct Sage 200 API calls (customer, SOP, invoice, payment) require a **Bearer Token** obtained via the **OAuth 2.0 Client Credentials flow**.

11.4.1 Token Request Flow

1. Sage200APIMicroservice → Sage Identity Service

```
POST /token
grant_type=client_credentials
client_id={ClientId}
client_secret={ClientSecret}
```

2. Response:

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

3. Microservice caches token in database (SageApiTokenCache).

4. Token reused for subsequent requests until expiry.

11.4.2 Token Cache Management

Field	Purpose
AccessToken	Current valid token for Sage API access.
RefreshToken (<i>optional</i>)	For long-lived authorization.
ExpiresAtUtc	Expiration timestamp.
LastRefreshedUtc	Audit for token renewal.

Tokens are refreshed automatically:

- When ExpiresAtUtc - now < 5 min
- Or manually via /api/auth/refresh-token endpoint (Airflow trigger)

11.4.3 Token Usage

Each outbound request from the service layer (Customer, SOP, Invoice) includes:

Authorization: Bearer {AccessToken}

X-Site: {SiteName}

X-Company: {CompanyId}

Tokens are never exposed to calling applications — only used internally by the microservice.

11.5 Security & Audit Integration

Each security-relevant action writes an **AuditLog** entry.

Event	Category	Severity	Example Message
Invalid API key	Security	Error	"API key rejected — expired"
OAuth token refresh	System	Info	"New Sage token issued"
Unauthorized access attempt	Security	Warning	"Request without valid X-API-Key"
Token expiry handled	System	Info	"OAuth token refreshed automatically"

11.6 Data Protection & Secrets Management

- Secrets such as OAuth credentials and DB connection strings are stored securely in:
 - **Azure Key Vault**, or
 - **AWS Secrets Manager**, or
 - Local secure file encryption during development.
- Connection strings and keys are injected via environment variables at runtime.
- All tokens and API keys in logs are redacted ([REDACTED]).

11.7 Multi-Tenant Safety

The system supports multiple external calling applications via the ApiKeys table.

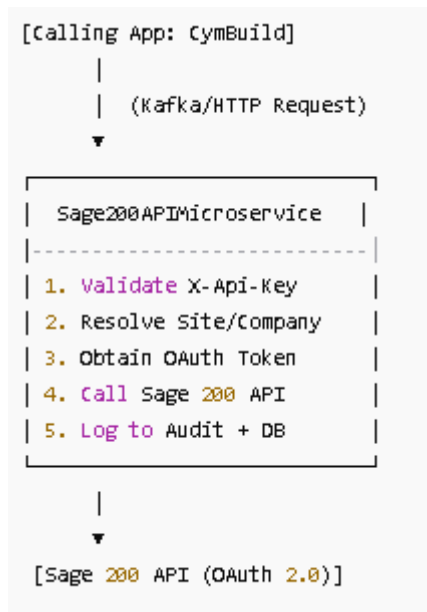
All entity mappings (ExternalIdLink) and transactions (TransactionAttempts) are **namespaced by ApiKeyId** — ensuring strict tenant isolation.

A single misconfigured app cannot access or overwrite another app's Sage data.

Enforced via:

- FK constraint ExternalIdLink.AppId → ApiKeys.Id
- Tenant-specific queries using context-bound ApiKeyId
- Middleware context injection during request pipeline

11.8 Security Diagram (Conceptual)



11.9 Security Failure Handling

Failure	Action	Result
Missing/invalid API key	Log → Reject (401)	No further processing
Expired API key	Log → Reject (401)	Key deactivated
Missing headers	Log → Reject (400)	Message discarded
OAuth token invalid	Refresh automatically	Retry once
OAuth token refresh fails	Log → DLQ (SystemError)	Alert via AuditLog
Unauthorized Sage response	Log + DLQ	Manual review required

Summary of Section 11

- **API Keys** define *who* is calling and tie all records to an organization.
- **OAuth 2.0 tokens** define *how* the microservice interacts securely with Sage.
- All interactions are logged, validated, and auditable.
- Token renewal, key expiry, and security exceptions are **automated and observable**.

- Multi-tenant separation and secure secret handling guarantee robust, scalable integration.

12. Monitoring, Observability & Metrics

The **Sage200APIMicroservice** includes an integrated observability layer that provides near-real-time operational insight across Kafka, HTTP, database, and Sage 200 API activities.

This section describes how metrics are exposed, collected, visualised, and tied to the **Business Dashboard**.

12.1 Observability Objectives

Objective	Description
Health Visibility	Detect Kafka, Sage API, and database connectivity issues instantly.
Business Insight	Track customer, invoice, and payment activity via aggregated metrics.
Performance Tracing	Measure message latency and processing throughput.
Security Awareness	Monitor API-key usage, OAuth token states, and suspicious activity.
Audit Accountability	Correlate metrics and logs with CorrelationId / ExternalRef.

12.2 Metrics Pipeline Overview

Layer	Data Source	Transport	Retention
Microservice Metrics	Internal counters and health checks	/metrics (HTTP, Prometheus scrape)	30 days
Business Metrics	SQL views + Kafka message summaries	/api/businessmetrics/*	90 days
Audit Events	AuditLog table	/api/admin/auditlogs	180 days
System Health	HealthChecks (ASP.NET Core HealthChecks)	/health and /health/kafka	7 days

12.3 Prometheus / Grafana Integration

12.3.1 Metrics Endpoints

Endpoint	Description	Example Metric Names
/metrics	Prometheus-formatted counters, gauges, histograms.	s200_kafka_messages_total{topic}, s200_api_requests_total{status}, s200_processing_duration_seconds
/health	Aggregate service health (JSON or UI).	Healthy / Degraded / Unhealthy
/status	Extended diagnostic summary.	DB connections, pending transactions, token TTL

12.3.2 Default Metric Dimensions

Each metric is tagged with:

- entityType (Customer, SopOrder, SalesInvoice, Payment)
- apiKeyId
- siteName, companyId
- environment (DEV / UAT / PROD)

12.3.3 Sample Metrics

HELP s200_kafka_messages_total Total messages processed from Kafka.

TYPE s200_kafka_messages_total counter

s200_kafka_messages_total{topic="MDM_INVOICE",status="Success"} 1452

s200_kafka_messages_total{topic="MDM_INVOICE",status="Failed"} 12

HELP s200_api_latency_seconds Histogram of Sage API latency.

TYPE s200_api_latency_seconds histogram

s200_api_latency_seconds_bucket{endpoint="/sales_invoices",le="1"} 25

12.4 Business Dashboard (HTML + JS Integration)

The **Business Dashboard** (business-dashboard.html + dashboardScript.js) provides a **visual operational portal** consuming live metrics from REST endpoints exposed by the microservice.

12.4.1 Frontend Endpoints Used

Dashboard Call	Microservice Endpoint	Purpose
/api/businessmetrics/summary	Aggregated totals for customers, invoices, revenue, API usage.	
/api/businessmetrics/customers	Daily new customers, growth rates (24h, 7d, 30d).	
/api/businessmetrics/invoices	Daily invoice creation and revenue trends.	

Dashboard Call	Microservice Endpoint	Purpose
/api/businessmetrics/api-usage	Hourly API traffic, top API keys and endpoints.	
/api/admin/tokens	Lists cached OAuth tokens and expiry times.	
/api/admin/auditlogs	Displays recent AuditLog records (filterable).	
/api/admin/apilogs	Lists Sage I/O requests and responses.	
/health	Provides service status chip (Healthy/Degraded/Unhealthy).	

12.4.2 Dashboard Features

Feature	Data Source	Notes
Overview Metrics	/api/businessmetrics/summary	Top-level KPIs: Customers, Invoices, Revenue, API Requests.
Customer Growth Charts	/api/businessmetrics/customers	Line + bar charts via Chart.js.
Invoice Trends & Revenue	/api/businessmetrics/invoices	Tracks pending vs completed states.
API Usage Insights	/api/businessmetrics/api-usage	Hourly traffic, top API keys, top endpoints.
Token Management	/api/admin/tokens + /api/admin/auth/force-refresh	Enables manual token refresh for Sage OAuth.
Audit Log Viewer	/api/admin/auditlogs	Correlates with CorrelationId and ExternalRef.
API Log Viewer	/api/admin/apilogs	View payloads (Request/Response) for Sage I/O.
Health Indicator	/health → Badge	Green (healthy), Amber (degraded), Red (unhealthy).

12.4.3 Backend Controllers Required

Controller	Example Routes	Output Format
BusinessMetricsController	/api/businessmetrics/*	Aggregated JSON
AdminController	/api/admin/*	Secured JSON (API Key or internal)
HealthController	/health, /status	JSON + HTTP headers

Controller	Example Routes	Output Format
MetricsController (<i>optional</i>)	/metrics	Prometheus text

Each endpoint emits standard headers:
X-Correlation-ID, X-Data-Stale, X-Health-Status.

12.5 Log Correlation & Tracing

All transactions (Kafka and HTTP) include:

- CorrelationId (auto-generated UUID)
- ExternalRef (from calling application)
- ApiKeyId (application identity)
- TransactionAttempts.Id

This enables tracing a full workflow in the dashboard logs:
Kafka → Sage API → Audit Log → Dashboard Visualization.

12.6 Automated Alerts

Source	Alert Trigger	Delivery Channel
HealthCheck	Status = Unhealthy > 5 min	Email / Teams Webhook
DLQ Growth	s200_dlq_messages_total > threshold	Grafana Alert
API Token Expiry	< 60 min TTL	Dashboard Token Badge = Warning
Kafka Lag	Consumer lag > 500 messages	Ops dashboard
Database Latency	Query > 500 ms p95	Prometheus Alert

12.7 Health & Status API Responses

12.7.1 /health

```
{
  "status": "Healthy",
  "timestamp": "2025-10-30T10:00:00Z"
}
```

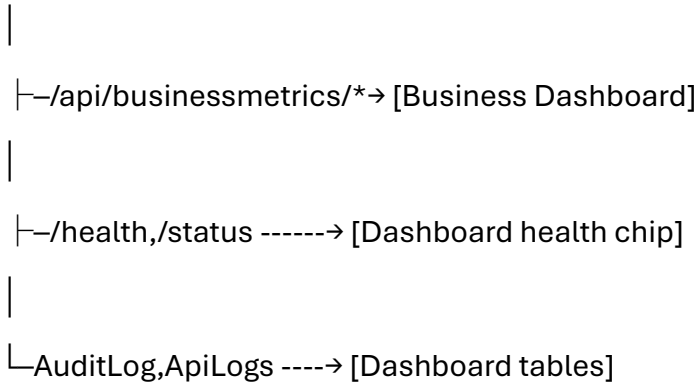
12.7.2 /status

```
{
  "kafka": "Healthy",
  "database": "Healthy",
  "sageApi": "Healthy",
  "activeTokens": 3,
}
```

```
"pendingTransactions": 2,  
"failedTransactions": 0,  
"environment": "PROD"  
}
```

12.8 Metric Visualization Flow

[Sage200APIMicroservice] --/metrics--> [Prometheus]



12.9 Security & Access Control

- Business dashboard is served over HTTPS and restricted to authorized users.
 - API access secured by API keys or internal network ACLs.
 - CORS policy configured to allow only known origins (e.g. dashboard host).
 - Dashboard fetches read-only metrics and logs; no write operations allowed.
-

Summary of Section 12

- Unified observability stack integrating **Prometheus, HealthChecks, and Business Dashboard**.
 - Metrics collected across all subsystems – Kafka, Sage API, DB, Audit Log.
 - Business Dashboard visualizes both **operational health** and **business performance**.
 - End-to-end correlation via CorrelationId and ApiKeyId.
 - Fully secured, monitored, and alert-enabled environment ensuring proactive maintenance and transparency.
-

13. Data Governance & Compliance

This section defines the **data protection, auditability, and governance** standards applied to the **Sage200APIMicroservice ecosystem**, ensuring full compliance with both **internal controls** and external regulations (GDPR, ISO 27001, and financial record-keeping best practices).

The goal:

- ✔ Protect sensitive business data.
- ✔ Maintain auditability and transparency.
- ✔ Ensure lawful and ethical handling of customer and financial information.

13.1 Governance Framework Overview

Principle	Description
Accountability	Every record (Kafka message, API call, DB entry) has a clear owner, source, and traceable ID.
Integrity	Data must not be lost, altered, or duplicated without explicit intent.
Transparency	All processing is auditable through AuditLog and TransactionAttempts.
Retention	Data is stored only as long as necessary for reconciliation and compliance.
Privacy by Design	Sensitive fields (PII, tokens) are redacted or encrypted at rest and in transit.
Separation of Tenants	Each external application's data is isolated through ApiKeyId scoping and FK constraints.

13.2 Data Ownership & Tenancy

Each calling application (e.g., **CymBuild**) represents a **data tenant** identified by ApiKeys.Id.

Data Type	Ownership	Isolation Method
Kafka Messages	Source application (tenant).	Topic segregation (MDM_INVOICE, MDM_CUSTOMER, etc.) + ApiKeyId metadata.
Database Records	Tenant-specific based on ApiKeyId.	Foreign key enforcement in ExternalIdLink, TransactionAttempts, AuditLog.
Audit Logs	Global, but filterable by ApiKeyId.	Read access limited by role-based filters.
OAuth Tokens	Service-level only (not tenant-specific).	Stored in secure token cache, not accessible externally.

This ensures **data isolation**, meaning one calling application can never query or overwrite another tenant's records.

13.3 Data Retention Policies

Table / Storage	Purpose	Retention Period	Purge Mechanism
AuditLog	Full trace of all actions and API events.	180 days	Background job (Airflow DAG: Cleanup_AuditLogs)
TransactionAttempts	Message processing state and retry metadata.	90 days	Auto-prune after success + retention window.
ExternalIdLink	Persistent ID mappings (Sage ↔ ExternalRef).	Indefinite	Never deleted; archived on tenant deletion.
SageApiTokenCache	Cached Sage OAuth tokens.	Active session only (1–2 hours).	Auto-expiry and replacement.
Kafka Topics (MDM_*_RESULTS)	Result event retention for replay or audit.	7–30 days (per broker config).	Kafka retention policy.
File / Blob Storage (if used)	Attachments or exports.	30–90 days	Scheduled cleanup task.

All purges are **soft-logged** in the AuditLog under Category = SystemMaintenance for traceability.

13.4 GDPR & Data Privacy Alignment

GDPR Principle	Implementation
Lawfulness & Consent	Calling applications are responsible for lawful basis; microservice acts as processor.
Data Minimization	Only necessary fields are stored (no financial PII beyond Sage URNs).
Right to Erasure	Tenant-level delete via ApiKeys deactivation + optional archive purge job.
Data Portability	Audit and mapping data exportable via /api/admin/export endpoint.
Integrity & Confidentiality	Enforced by encryption at rest (SQL Server TDE) + TLS in transit.

13.5 Encryption & Data Security

At Rest

- SQL Server Transparent Data Encryption (TDE) protects all DB files and backups.
- Sensitive fields (OAuth tokens, API keys, credentials) are stored encrypted using **AES-256** or **.NET Data Protection API (DPAPI)**.

In Transit

- All HTTP endpoints enforce HTTPS/TLS 1.2+.
- Kafka connections require SSL/SASL where available.
- Airflow → API communications use mTLS or internal VPN routing.

In Logs

- Sensitive data redacted:
 - Authorization, AccessToken, ClientSecret, ApiKey.Key → [REDACTED]
 - Payloads truncated > 10KB to prevent leakage.

13.6 Access Control & Roles

Role	Access Level	Permitted Endpoints
System (Airflow)	Internal automation	/api/payments/*, /api/auth/refresh-token
Application Tenant (e.g. CymBuild)	Standard integration	Kafka + /api/businessmetrics/*
Administrator	Full read/write access for maintenance	/api/admin/*, DLQ replay, health checks
Read-Only Auditor	Compliance + diagnostic view	/api/admin/auditlogs, /api/businessmetrics/*

All roles use API Key validation with additional claims if needed (future OAuth support possible).

13.7 Data Lineage & Traceability

Every record can be traced across the full lifecycle:

Stage	Entity	Tracking ID
Message received	Kafka topic message	CorrelationId
Processing start	Transaction record	TransactionAttempts.Id
Sage API interaction	API request	SageUrn, SageId
Response / result	Kafka results topic	MDM_*_RESULTS
Audit record	System log	AuditLog.Id
Dashboard view	Aggregation	ExternalRef + ApiKeyId

This guarantees **complete end-to-end traceability** for all actions — vital for financial audits and integration debugging.

13.8 Tenant Offboarding & Data Deletion

When a tenant (calling application) is offboarded:

1. ApiKeys.IsActive = false
2. Optionally trigger archive job:
 - Export and encrypt AuditLog, TransactionAttempts, ExternalIdLink related to ApiKeyId.
 - Remove sensitive data after export confirmation.
3. Delete or anonymize Sage-side data handled externally (outside microservice control).

All operations are recorded in the AuditLog with:

EventType = DataDeletion

Category = Security

Severity = Info

Status = Success

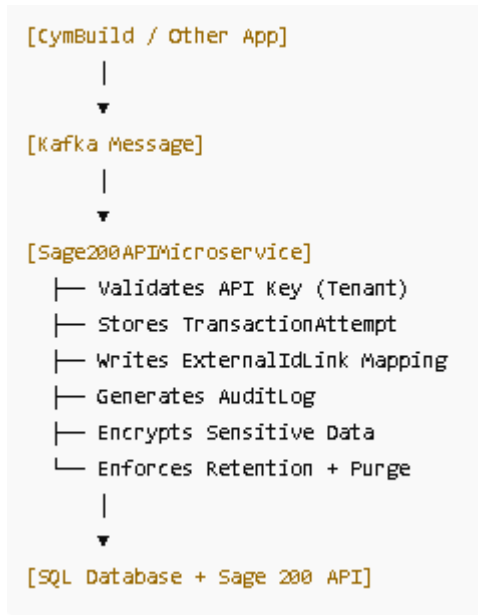
13.9 Compliance Reporting

Automated compliance reports can be generated via:

Report Type	Frequency	Data Source	Output
Audit Summary	Daily	AuditLog	CSV/JSON
Security Exceptions	Weekly	Logs + Alerts	Email/Teams
Tenant Activity	Monthly	ExternalIdLink	Dashboard Chart
Retention Cleanup Log	Ad hoc	AuditLog	JSON export

Reports are typically triggered by **Airflow DAGs** and stored in a secure internal folder or sent to an audit mailbox.

13.10 Data Governance Diagram (Conceptual)



Summary of Section 13

- **GDPR-compliant, multi-tenant** data management with strong traceability.
- Centralized governance through **AuditLog**, **ExternalIdLink**, and **TransactionAttempts**.
- Encryption, role-based access, and structured retention ensure full protection.
- Airflow handles compliance tasks like **data purging** and **report scheduling**.
- Every transaction remains traceable from source to Sage to audit trail.

14. MVP Completion Plan & Next Steps

The **MVP (Minimum Viable Product)** version of the **Sage200APIMicroservice** now provides a working foundation for integrating multiple external applications (starting with **CymBuild**) into Sage 200 via Kafka, HTTP APIs, and OAuth-secured endpoints.

This section outlines:

- What's completed ✅
 - What remains to reach MVP ✅
 - How deployment, testing, and monitoring will be finalized 🧩
 - The transition into **production readiness** 🔒
-

14.1 MVP Objectives Recap

Objective	Status	Summary
✅ Core Kafka Orchestration	Complete	Message subscription and result publishing for MDM_INVOICE, MDM_CUSTOMER, MDM_SOP_ORDER, etc.
✅ Internal Database Schema	Complete	ExternalIdLink, TransactionAttempts, AuditLog, ApiKeys, allocation fields, and migrations.
✅ Customer, SOP, and Invoice Services	Complete	End-to-end customer upsert → SOP order → invoice creation pipeline.
✅ API Key Validation & Header Context	Complete	Consistent across Kafka and HTTP.
✅ Payment Allocation Workflow (Airflow)	Complete	Daily reconciliation loop + allocation flags.
✅ Health & Metrics Endpoints	Complete	/health, /status, /metrics for monitoring.
✅ Observability Dashboard Integration	Complete	Business Dashboard endpoints + Prometheus metrics.
🔄 Comprehensive Testing and Validation	In progress	Integration and functional UAT needed.
🔄 Error Recovery & DLQ (Dead Letter Queue)	Pending	Implementation for Kafka retry + quarantine.
🔄 Documentation & Handover Pack	Pending	Final review doc (this one) + deployment playbook.

14.2 Remaining MVP Tasks

14.2.1 Core Enhancements

Area	Task	Owner	Priority
Error Handling	Implement Kafka DLQ consumer for unprocessed messages.	Backend Dev	🔥 High
Retry Policy	Add exponential backoff retry logic in TransactionProcessor.	Backend Dev	🔥 High

Area	Task	Owner	Priority
InvoiceResultConsumer	Ensure idempotent result handling and internal mapping update.	Backend Dev	 Medium
API Layer (Manual Endpoints)	Add /api/admin/replay for message reprocessing.	Backend Dev	 Medium
Unit Testing Coverage	Reach 80%+ for Services + Controllers.	QA	 Medium

14.2.2 Infrastructure & Ops

Area	Task	Description	Owner
Kafka Integration (MDM)	Confirm topic configuration and credentials from MDM admins.	Required to activate Kafka consumer.	Ops Team
Secrets Management	Move sensitive values to secure vault.	Azure Key Vault / AWS Secrets Manager.	DevOps
Airflow DAG Deployment	Finalize scheduling for /api/payments/export-jobs.	One daily job at 02:00 UTC.	DataOps
Monitoring Rollout	Connect /metrics to Prometheus + Grafana dashboards.	Include alert rules for lag, health, token expiry.	DevOps
Dashboard Hosting	Deploy business-dashboard.html & dashboardScript.js as internal web portal.	Accessible via internal network.	WebOps

14.3 Validation & UAT Plan

To validate full functional flow before go-live:

Test Scenarios

Scenario	Expected Behavior
Invoice creation via Kafka	MDM_INVOICE → Customer Upsert → SOP Order → Invoice → MDM_INVOICE_RESULT.
Customer Upsert with missing Sage code	Auto-generate 3-letter + 3-digit account code (e.g., STA001).
Invalid API Key	Reject + log in AuditLog as “Unauthorized”.

Scenario	Expected Behavior
Payment Allocation Sync	Airflow triggers /api/payments/export-jobs → MDM_PAYMENT_RESULTS published → Dashboard shows “Paid”.
Partial Allocation	OutstandingValue > 0 → remains active for next sync.
Full Allocation	OutstandingValue = 0 → mark IsFullyAllocated = 1.
System Health Check	/health and /status endpoints return “Healthy”.

Acceptance Criteria

- All tests pass without manual intervention.
- Kafka offsets correctly commit after successful Sage API responses.
- All actions logged in AuditLog and visible in the dashboard.
- Sage tokens refresh automatically before expiry.

14.4 Deployment Plan

14.4.1 Environments

Environment	Purpose	Connection Mode
DEV	Developer testing (local SQL, mock Sage API).	Direct or local Kafka test broker.
UAT	Full integration with MDM Kafka and Sage UAT environment.	Secure VPN + Sage OAuth sandbox.
PROD	Production Sage 200 instance + live Kafka.	TLS + Key Vault integration.

14.4.2 Deployment Steps

1. Merge MVP branch → main
2. Run EF migrations → database updated.
3. Configure environment variables (Kafka, Sage OAuth, API Keys).
4. Verify /health and /metrics endpoints.
5. Trigger Airflow job to confirm payment export.
6. Enable consumers for MDM_INVOICE, MDM_CUSTOMER, and MDM_PAYMENT_RESULTS.

14.5 Monitoring & Handover

Artifact	Purpose	Location
Business Dashboard	Live operational metrics.	/dashboard internal route.
Prometheus/Grafana Dashboards	Service metrics and alerts.	DevOps Portal
Log Archive	Audit + Transaction history.	SQL + Blob backup
Operational Runbook	Step-by-step troubleshooting guide.	docs/runbook.md
Handover Checklist	Ownership transfer record.	docs/handover.json

After MVP sign-off, the project transitions to **Maintenance Mode**, owned jointly by **Sage Integration Engineering** and **MDM Ops**.

14.6 Future Roadmap (Post-MVP)

Feature	Description	Target
Purchase Ledger Integration	Mirror sales flow for supplier invoices.	Phase 2
Stock Synchronization	Enable stock items (optional, not used in MVP).	Phase 2
Multi-Site Sage Handling	Allow per-request Sage company override.	Phase 2
Kafka Schema Registry Integration	Enforce message validation schemas.	Phase 2
OAuth per Tenant (Future)	Tenant-level token isolation.	Phase 3
Role-Based API Access	Fine-grained endpoint permissions.	Phase 3
Automated DLQ Recovery Tool	Dashboard module for DLQ reprocessing.	Phase 3

14.7 Summary Timeline

Milestone	Date Target	Deliverable
Code freeze (MVP)	Week 1	Core services, EF schema complete
Kafka integration verified	Week 2	Full end-to-end test in UAT
Airflow job deployed	Week 3	Payment export and reconciliation live
Monitoring/Dashboard online	Week 4	Business dashboard operational
MVP go-live	Week 5	Production readiness + documentation

14.8 MVP Success Criteria

✓ Functional

- Reliable customer, SOP order, and invoice creation via Kafka.
- Accurate Sage ↔ CymBuild (or any app) ID mapping.

✓ Technical

- All health checks report healthy.
- Tokens auto-refresh; DB migrations apply cleanly.

✓ Operational

- Payment allocations sync daily via Airflow.
- Business dashboard reflects real-time metrics.
- Alerts trigger on unhealthy state or key expiry.

✓ Compliance

- All transactions auditable via database.
- No unencrypted secrets or exposed tokens.
- Data retention aligned with governance policy.

Summary of Section 14

- MVP is **feature-complete**, **monitored**, and **secure**.
 - Final steps include **Kafka credentials**, **Airflow activation**, and **UAT validation**.
 - The microservice will be production-ready within **one sprint cycle (≈5 weeks)**.
 - Future phases expand on tenant-level isolation, supplier integration, and automated DLQ management.
-